

Claude Code Alternatives: A Comprehensive Survey of AI Coding Agents in 2026 (September 9 Refresh)

September 9, 2026

Table of Contents

1	Introduction	2
2	The AI Coding Agent Taxonomy	3
3	CLI-Native Tools	4
3.1	Aider	4
3.2	OpenCode	7
3.3	Goose (by Block)	9
3.4	Gemini CLI (by Google)	10
3.5	Plandex	11
3.6	SWE-agent	12
3.7	OpenAI Codex CLI	13
3.8	Amp (Sourcegraph)	14
3.9	Deprecated and Unverified Tools	15
3.10	Mentat	15
3.11	Qwen Code and Kimi CLI	16
4	IDE Extensions and Hybrid Tools	16
4.1	Cline (formerly Claude Dev)	16
4.2	Continue.dev	18
5	Dedicated AI IDEs	20
5.1	Cursor	20
5.2	Windsurf (now Devin Desktop, Cognition AI)	21
5.3	Zed AI	22
5.4	Google Antigravity	23
5.5	AWS Kiro IDE and CLI	24
6	Cloud and Web Platform Agents	25
6.1	Grok Bot (xAI)	25
6.2	OpenHands (formerly OpenDevin)	27
6.3	Devin (by Cognition AI)	28

7	Commercial AI Coding Assistants	29
7.1	GitHub Copilot with Agent Mode	29
7.2	Amazon Q Developer (formerly CodeWhisperer) – IDE transition to Kiro . .	30
7.3	JetBrains AI Assistant	31
7.4	Tabnine	32
7.5	Supermaven – discontinued, folded into Cursor	32
8	Cross-Cutting Analysis	33
8.1	Provider Flexibility Analysis	33
8.1.1	OpenRouter Compatibility	33
8.1.2	Local LLM Support (Ollama / LM Studio)	37
8.2	Feature Comparison Matrix	43
8.3	Extensibility Deep Dive	47
8.3.1	Claude Code’s Extension Architecture	47
8.3.2	How Alternatives Compare	48
8.3.3	The AGENTS.md Open Standard	49
8.3.4	What This Means for Migration	49
8.4	Cost Analysis	49
8.4.1	Defining “Heavy Usage”	50
8.4.2	API Cost Estimates (Direct Provider, as of August 31, 2026)	50
8.4.3	Per-Tool Cost Summary	51
8.4.4	Cost Migration Strategy	59
9	Recommendations	59
9.1	If you want the closest CLI experience to Claude Code	59
9.2	If you want the best IDE-integrated alternative	60
9.3	If cost is the primary constraint	60
9.4	If you need persistent cross-application or asynchronous work	61
9.5	If MCP server investments are critical	61
9.6	If you need extensibility similar to Claude Code’s skills+hooks	61
9.7	Migration strategy	62
10	References	62

1 Introduction

Claude Code has established itself as one of the most capable agentic coding tools available: it runs in the terminal and supported editors, takes high-level natural-language instructions, autonomously edits multiple files, executes shell commands, runs tests, and iterates until the task is done. Its extensibility system – skills, hooks, and MCP server support – allows deep customisation of its workflow. For heavy users, a Max plan (**\$100/month for Max 5x, \$200/month for Max 20x**) can be good value relative to metered API pricing, but it is not unlimited: Max has a five-hour session limit and a separate weekly limit, and Anthropic may

apply additional caps. Limits are shared across Claude, Claude Code, and Claude Desktop. The plan landscape for AI tools changes rapidly, and a prudent engineer should understand the full landscape of alternatives before needing them.

This document surveys the landscape of AI coding agents available as of **September 9, 2026**: open-source CLI tools, IDE extensions, dedicated AI IDEs, cloud platform agents, and commercial assistants. For each, it covers architecture, provider flexibility, MCP/extensibility support, and realistic cost.

A note on methodology and provenance: This document was originally drafted from an AI model’s training-data snapshot (accurate as of approximately August 2025), then refreshed via live web research in July and August 2026. The August 31 pass rechecked the most volatile claims against first-party product pages, including Claude and Codex limits, Amp’s subscription and Orb pricing, Cline’s terminal/plugin/hook support, Goose’s current repository, Kiro’s unified IDE/CLI/Web architecture, Gemini CLI’s transition status, and current DeepSeek pricing. The September 9 pass added Grok Bot and checked its launch, architecture, controls, privacy requirements, and current access against first-party xAI pages. Dated facts carry a date at the point where the distinction matters, with corresponding sources in the References section.

Freshness note: In this field, some sections can age in weeks, not quarters. Treat pricing, benchmark rankings, and model-version statements as snapshots tied to their stated dates.

How to read this document: If you want the fastest path to a conclusion, jump to the [Feature Comparison Matrix](#), the [Provider Flexibility Analysis](#), and the [Recommendations](#). The deep-dive sections are there for when you need to evaluate a specific tool seriously.

This survey covers tools that were verifiable and actively maintained as of September 2026. It does not cover tools no longer in active development, purely GUI-based editors with no API or CLI surface, or general-purpose LLM interfaces that happen to accept code. Cloud IDE platforms (Replit, Gitpod, etc.) are out of scope unless they offer a dedicated coding-agent mode. Where a claim could not be verified, it is flagged.

2 The AI Coding Agent Taxonomy

Before comparing tools, it helps to understand the four distinct categories that have emerged. This taxonomy comes from Artificial Analysis’s coding agent classification (verified):

CLI Tools run entirely in the terminal. They take instructions, edit files, run commands,

and loop autonomously. This is the category Claude Code belongs to. Other members: Aider, OpenCode, Goose, Gemini CLI (legacy/enterprise path), Antigravity CLI, Plandex, SWE-agent, Qwen Code, Kimi CLI, OpenAI Codex CLI, Amp (hybrid local/cloud), and GitHub Copilot CLI (distinct from the IDE-based Copilot covered under IDE Extensions below).

IDE Extensions augment an existing editor (primarily VS Code or JetBrains). They have full access to the editor’s language server, refactoring tools, and UI, but are less suitable for scripted or headless workflows. Members: Cline, Continue.dev, GitHub Copilot, Amazon Q Developer, Tabnine, JetBrains AI Assistant.

Dedicated AI IDEs are entire editors rebuilt around AI-first workflows. They typically fork VS Code and add deeper AI integration than an extension permits (Zed is the exception, built from scratch in Rust). Members: Cursor, Windsurf (now Devin Desktop, Cognition AI), Zed AI, AWS Kiro IDE, Google Antigravity 2.0.

Cloud Platform Agents run primarily in the cloud or a sandboxed environment (Docker). They expose a web or app-based interface and are designed for longer-running autonomous tasks, often with their own execution environments. Members: Grok Bot, OpenHands, Devin, Manus, Jules, Genie. Grok Bot sits at the broadest edge of this category: it is a persistent cross-application computer-use agent that can do coding work, not a coding-specific terminal agent.

A Claude Code user primarily cares about the CLI tools category, but the IDE and cloud categories contain tools capable enough to be worth understanding as alternatives – especially if your workflow includes time in an editor.

3 CLI-Native Tools

3.1 Aider

What it is: Aider is the most mature and architecturally similar open-source alternative to Claude Code. It is a terminal-based pair-programming tool that works directly with your existing git repository, takes natural-language instructions, autonomously edits multiple files, and commits changes.

Author and licence: Created by Paul Gauthier. Apache-2.0 licence. Open source at github.com/Aider-AI/aider.

Community size: Roughly mid-40k to low-50k GitHub stars (late-July 2026 snapshot).

This is among the largest in the open-source CLI coding agent category. Star counts and commit velocity shift quickly, so treat this as a dated snapshot rather than a durable ranking.

Architecture: Aider is written in Python and uses the `litellm` library as its LLM abstraction layer, which means it can talk to virtually any LLM provider that `litellm` supports. Its edit format is model-dependent: `diff` uses search/replace blocks, while `udiff` uses a unified-diff-derived format; `whole` rewrites the entire file. Aider chooses a suitable format for the model unless you override it. Accepted changes can then be committed automatically. Aider supports several edit modes:

- **diff** – search/replace blocks; the common default for supported models
- **udiff** – unified-diff-derived edit blocks
- **whole** – the model rewrites the entire file (less efficient, sometimes more reliable for small files)
- **architect mode** – a two-step process where one model plans the changes and a second cheaper model applies them, reducing cost

Installation:

```
pip install aider-chat
# or with uv
uv tool install aider-chat
```

LLM provider support: Aider supports all major providers via `litellm`, including:

- Anthropic (Claude family – `claude-sonnet-5`, `claude-opus-5`, `claude-haiku-4-5-20251001`)
- OpenAI (current GPT-5.6 and Codex model aliases exposed by the installed Aider/litellm release)
- Google (`gemini-3.1-pro-preview` via Vertex AI or AI Studio)
- AWS Bedrock
- Azure OpenAI
- OpenRouter (verified – full setup documented at `aider.chat/docs/llms/openrouter.html`)
- Ollama for local models
- LM Studio
- Groq, Mistral, Cohere, and many others

OpenRouter setup (verified):

```
export OPENROUTER_API_KEY=sk-or-...
aider --model openrouter/anthropic/claude-sonnet-5
# or any OpenRouter-hosted model:
aider --model openrouter/deepseek/deepseek-v4-pro
aider --model openrouter/google/gemini-3.1-pro-preview
```

Local LLM setup (Ollama):

```
# Start Ollama with a coding model first (example tag; verify the current model catalogue)
ollama pull qwen2.5-coder:7b
```

```
# Then run Aider
aider --model ollama/qwen2.5-coder:7b
```

Agentic capabilities: Aider can run shell commands via the `/run` command and in `--auto-run` mode will execute suggested commands automatically. It maintains a context of added files and can be instructed to add more mid-session. It supports `/web` for fetching URLs into context and can integrate with test runners. The workflow is: add files to context → give instruction → model proposes diffs → Aider applies and commits → repeat.

Git integration: Every accepted change is automatically committed. This is tightly integrated – you can see the full history of AI changes in `git log`. There is also an `--auto-commits` flag to control this behaviour.

Shell execution: Via `/run <command>` or `--auto-run`. Aider can also be configured with `--test-cmd` to run tests automatically after each change.

MCP support: Still absent as of late July 2026. RFC #4506 proposing native MCP support remains open, and an exploratory PR (#3937) was closed without shipping. This remains a meaningful gap compared to Claude Code and other MCP-native tools.

Extensibility: Aider has limited plugin architecture compared to Claude Code. Configuration is via `.aider.conf.yml` and environment variables. There is no equivalent to Claude Code’s skills or hooks system. What it lacks in extensibility it compensates for in simplicity and reliability.

Modes:

- **code** – the default mode, edits files
- **architect** – planning model + editing model split
- **ask** – ask questions about code without editing
- **help** – help with Aider itself

Cost model: Open source, free to use. You pay only for the LLM API calls you make. The cost of a session depends on the selected model, context, and number of turns; use the provider’s current rate card rather than a fixed per-session estimate. With OpenRouter you can route to cheaper models.

Strengths: - Most CLI-native and architecturally similar to Claude Code of all open-source options - Excellent git integration – every change is tracked - Wide provider support via litellm - OpenRouter support is well-documented and works reliably - Mature, battle-tested codebase with high benchmark performance - Supports local LLMs via Ollama - Very low overhead – just Python and an API key

Weaknesses: - No MCP support, and no committed timeline for one (RFC open, prior PR abandoned) - No skills/hooks system – less extensible than Claude Code - Context management is manual (you explicitly `/add` files) - Less capable at long multi-step autonomous tasks compared to Claude Code - No built-in web search or browser use - Development cadence has slowed relative to faster-moving competitors

3.2 OpenCode

What it is: OpenCode is an open-source coding agent with a rich terminal UI (TUI), a client/server architecture, and support for a very wide range of LLM providers.

Author and licence: Originally built by the SST team; the project rebranded in early 2026 under a new organisation, Anomaly Inc., and the repository moved from `sst/opencode` to `anomalyco/opencode`. Open source.

Community size and activity: OpenCode has a large, fast-moving open-source community and a broad provider ecosystem. Exact star counts, release numbers, and provider totals change quickly; use the current repository and provider documentation rather than a hard-coded snapshot.

Architecture: Current OpenCode is primarily a TypeScript/Bun application with a client/server architecture and a full terminal user interface (TUI). The repository also contains a small Rust component, but describing the product as a Go/Bubble Tea binary is obsolete. It integrates with the AI SDK and Models.dev for LLM provider abstraction. It also supports Language Server Protocol (LSP) for code intelligence, meaning it can provide type-aware, semantically accurate code context rather than just raw file contents. Multi-session support allows you to maintain separate contexts for different tasks.

Installation:

```
# via npm (cross-platform)  
npm install -g opencode-ai
```

```
# or directly via binary release  
curl -fsSL https://opencode.ai/install | bash
```

LLM provider support: OpenCode supports 75+ LLM providers (verified) including:

- OpenAI (ChatGPT Plus/Pro can be authenticated from the provider picker)
- Google (Gemini via AI Studio or Vertex)
- AWS Bedrock
- Azure OpenAI
- OpenRouter
- Ollama (local models)

- LM Studio
- Grok (xAI)
- And many more via the AI SDK / Models.dev integration

This is the broadest provider flexibility of any CLI tool in the category.

Important change – subscription access: OpenCode’s providers documentation says that previous Claude Pro/Max plugins are no longer bundled as of v1.3.0 and are explicitly prohibited by Anthropic. The same documentation lists ChatGPT Plus/Pro, GitHub Copilot, and GitLab Duo as zero-setup subscription providers. For Claude, use an Anthropic API key unless you have verified the current authorization path and its terms; do not treat the old bundled-plugin route as supported.

OpenRouter and Ollama: Both confirmed as supported providers. Configuration is via `~/.config/opencode/opencode.json` or a project-level `opencode.json`.

Agentic capabilities: OpenCode can read and edit files, run shell commands, and iterate autonomously. The TUI provides a chat interface that feels more like a dedicated application than a REPL. LSP integration means it can navigate symbol definitions, find references, and understand project structure at the type level.

MCP support: OpenCode supports MCP servers. This is a significant advantage over Aider for users who have invested in an MCP ecosystem.

Cost model: Open source, free to use. Your cost depends on the selected provider: API billing, an eligible subscription, or an optional OpenCode Zen/Go plan.

Strengths: - Broadest LLM provider support of any CLI tool (75+ endpoints verified) - Rich TUI with a more polished interactive experience than most CLI tools - LSP integration for semantic code understanding - MCP support - Extremely active development, now with a large mainstream user base - Client/server design supports multiple front ends, including the TUI and a beta desktop application - Native ChatGPT subscription support

Weaknesses: - Relatively new compared to Aider – less battle-tested - TUI approach is slightly less scriptable than a pure REPL like Aider - Claude subscription access needs particular care: the old bundled plugin route is prohibited by Anthropic and is no longer shipped - Org/rebrand churn (`sst/opencode` → `anomalyco/opencode`) means older documentation and links may be stale

3.3 Goose (by Block)

What it is: Goose is an open-source, on-device AI coding agent developed by Block (formerly Square, the company associated with Jack Dorsey). It is available as a CLI, a desktop application, and an API. Its primary differentiator is an extension system built natively on MCP.

Author and licence: Developed by Block, Inc. and now maintained in the `aaif-goose/goose` project. Apache 2.0 licence. The current repository is `github.com/aaif-goose/goose`; older links to `block/goose` are stale.

Architecture: Goose is a native Rust agent available as a desktop app, CLI, and API. Agent execution runs on your machine; model calls go wherever you configure them, including local Ollama. Its extension system is built directly on MCP rather than a Goose-proprietary format – MCP is a core architectural pillar of the tool, not a bolted-on integration. Goose documents 70+ extensions and broad compatibility with the MCP ecosystem, so existing MCP investments generally carry over directly.

Installation:

macOS, Linux, or Windows

```
curl -fsSL https://github.com/aaif-goose/goose/releases/download/stable/download_cli.sh | bash
```

LLM provider support: Goose is model-agnostic and supports multiple provider configurations simultaneously, with 15+ documented provider integrations including Anthropic, OpenAI, Google, Groq, and Ollama. It does not lock you to a single provider and allows per-task model configuration.

Local LLM support: Confirmed support for Ollama – this is a core feature of the on-device design, not a partial or unverified claim.

Agentic capabilities: Goose can edit files, run shell commands, use its extension system to call external tools, and iterate autonomously on tasks. It has a particularly strong story for DevOps and infrastructure tasks.

MCP support: Confirmed and central to the architecture, not a hedged or disputed claim. Goose’s extension system is built on MCP and documents 70+ extensions. Goose is part of the Linux Foundation’s Agentic AI Foundation; that governance relationship is separate from the practical question of whether a given MCP server’s transport and authentication work in Goose.

Extensions: The extension system is the key differentiator, and it is MCP-native: extensions are effectively MCP servers, giving Goose access to the same growing ecosystem Claude Code and other MCP-compatible tools draw on.

Strengths: - Strong corporate backing from Block – less likely to be abandoned - On-device execution philosophy – good for privacy - MCP-native extension system with 70+ documented extensions and broad MCP ecosystem compatibility - Desktop app available for non-terminal users - Apache 2.0 licence - Model flexibility across 15+ providers - Part of the Linux Foundation’s Agentic AI Foundation ecosystem

Weaknesses: - Less pure CLI tool than Aider – the desktop app orientation means some rough edges in headless terminal use - Younger MCP ecosystem integration story than Claude Code’s, despite the architecture being sound

3.4 Gemini CLI (by Google)

What it is: Gemini CLI is Google’s open-source terminal AI agent, released in mid-2025. It is powered by the Gemini model family. Google announced a transition to Antigravity CLI: from June 18, 2026, Gemini CLI stopped serving individual Google AI Pro, Google AI Ultra, and free Gemini Code Assist accounts, while enterprise Gemini Code Assist and API/Vertex paths remain supported.

Author and licence: Google Cloud/Developer Products (not Google DeepMind, contrary to earlier reporting). Apache 2.0 licence. Open source at github.com/google-gemini/gemini-cli.

Community size: Gemini CLI saw extremely rapid adoption after launch – it became one of the fastest-growing GitHub repositories in history within weeks of release.

Architecture: Gemini CLI is a Node.js-based CLI agent. It uses Gemini’s Pro-tier model by default, which provides a very large context window – among the largest of any CLI coding tool. This means you can load entire codebases into context rather than managing file additions manually.

Installation:

```
npm install -g @google/gemini-cli  
gemini
```

LLM provider support: Gemini CLI is primarily designed for Google’s Gemini models (gemini-3.1-pro-preview, gemini-3.5-flash, and the model aliases exposed by the installed release). It is not model-agnostic in the same way as Aider or OpenCode.

Access and quotas: Google’s transition announcement says individual free, Pro, and Ultra access moved to Antigravity CLI on June 18, 2026. Enterprise Gemini Code Assist licences remain supported, and Gemini CLI remains available through API-key and Vertex AI authentication. A Gemini CLI plans page still displays a legacy-looking individual free tier, so treat that page as inconsistent with the transition announcement and test the actual sign-in path before relying on Gemini CLI for individual use.

MCP support: Gemini CLI supports MCP servers, making it one of the CLI tools with confirmed MCP integration alongside Claude Code and Goose. Configuration is via a `~/.gemini/settings.json` file.

Extensions: Gemini CLI has an extensions mechanism that allows adding capabilities beyond the default file editing and shell execution.

Agentic capabilities: Gemini CLI can read and edit files, run shell commands, search the web, and iterate autonomously. Its large context window means it is uniquely capable at tasks involving entire-codebase understanding.

Cost model: Individual Google-account access has moved to Antigravity CLI. Gemini CLI remains usable for enterprise Gemini Code Assist accounts and through Gemini API/Vertex AI authentication; those paths have their own quotas, billing, and data-use terms.

Strengths: - Large context window – can load entire codebases - MCP support - Open source (Apache 2.0) - Google backing means long-term support likely - Very active development

Weaknesses: - Locked to Gemini models – no OpenRouter or third-party model support - Individual consumer access is no longer the supported Gemini CLI path; migrate to Antigravity CLI - Gemini models, while capable, may not match Claude’s code quality for your specific use cases - Less battle-tested than Aider - Node.js dependency

3.5 Plandex

What it is: Plandex is an open-source CLI agent with a distinctive planning-first approach. Rather than immediately executing changes, it plans multi-file, multi-step tasks and builds up pending changes for user review before applying.

Author and licence: Created by Dane Schneider (@danenania). MIT licence. Open source at github.com/plandex-ai/plandex. Roughly mid-teens-thousands GitHub stars as of late July 2026, currently at v2.2.1.

Architecture: Plandex is written in Go and uses a client-server architecture. The server can be run locally or self-hosted on a cloud provider. Changes are accumulated in a “pending changes” buffer that you can review, modify, or reject before applying to the working tree. This makes Plandex the most review-oriented tool in the category – it assumes you want to understand what’s happening before it happens.

Installation:

```
# Install client
curl -sL https://plandex.ai/install.sh | bash

# Use cloud server (free tier available)
```

```
plandex sign-in
```

```
# Or self-host the server
```

```
docker-compose up -d # with the plandex-server repo
```

LLM provider support: Plandex supports OpenAI models directly and OpenAI-compatible APIs, which includes many providers. OpenRouter support via the OpenAI-compatible endpoint is possible.

Self-hosting: A key differentiator – you can run the entire Plandex server yourself, meaning no data leaves your infrastructure.

Agentic capabilities: Plandex is specifically designed for large, multi-step tasks: “implement this feature across 20 files.” Its planning mechanism handles long sequences of related changes better than tools that tackle each change independently.

Strengths: - Planning-first approach is excellent for large refactors - Self-hosting option for data sovereignty - Review buffer means less risk of unexpected changes - Good at long-horizon multi-file tasks

Weaknesses: - Planning overhead makes it slower for quick edits - Less interactive than Aider or OpenCode - Client-server architecture adds complexity - LLM provider support less broad than Aider/OpenCode - Smaller community

3.6 SWE-agent

What it is: SWE-agent is a research tool from Princeton NLP lab, designed primarily for automated software engineering on GitHub issues. It achieves high scores on the SWE-bench benchmark. It is less a daily-driver tool and more a demonstration of what autonomous agents can accomplish on discrete bug-fix tasks.

Author and licence: Princeton NLP group. MIT licence. github.com/SWE-agent/SWE-agent.

Architecture: SWE-agent uses a Docker-based sandboxed environment. The Agent-Computer Interface (ACI) pattern it introduces provides structured tools for file navigation, editing, and execution in a way that is particularly suited for benchmark tasks. Its strongest benchmark claims in this document are date-stamped to early 2026 and should be treated as historical snapshots, not permanent rankings. Active development in this lineage has shifted to a sibling project: **mini-swe-agent**, a ~100-line Python reimplement of the same core idea. If you’re evaluating this family today, check both projects and verify current SWE-bench standings before making a decision.

Intended use: Automated bug-fixing on GitHub issues, especially in batch/CI contexts.

Not designed for interactive development sessions.

Strengths: - Extremely high benchmark performance - Docker sandboxing for safe execution
- Good for automated CI/CD pipelines

Weaknesses: - Not a daily driver – requires Docker, not interactive - Research-oriented design means rough UX edges - Slower than interactive tools - Not designed for the ad-hoc, exploratory coding sessions that Claude Code excels at - Primary development momentum has moved to mini-swe-agent; SWE-agent proper sees less active iteration

3.7 OpenAI Codex CLI

What it is: Codex CLI is OpenAI’s open-source terminal coding agent, open-sourced April 16, 2025. It is OpenAI’s direct answer to Claude Code and Gemini CLI: a terminal-native agent that reads and edits files, runs shell commands, and iterates autonomously, with tight integration into ChatGPT’s subscription plans.

Author and licence: OpenAI. Open source at github.com/openai/codex. Roughly high-five-figures to low-six-figures GitHub stars (late-July 2026 snapshot); the codebase is 96.5% Rust.

Architecture: Codex CLI is a Rust binary, giving it a fast startup and low overhead compared to Node.js- or Python-based competitors. It defaults to OpenAI’s own hosted models but also supports local model backends.

Installation:

```
npm install -g @openai/codex  
# or via the standalone Rust binary release  
codex
```

Model support: The current Codex subscription lineup is the GPT-5.6 family: Sol, Terra, and Luna. Sol is the high-capability choice, Terra balances capability and allowance, and Luna stretches usage furthest. Available models depend on plan and may change, so `/model` and `/status` are more reliable than a hard-coded model name in a long-lived guide.

Local model support: Codex CLI can run against local models via an `--oss` flag, with documented Ollama and LM Studio providers. This gives it a genuine offline/local-only mode, something Gemini CLI lacks entirely.

Bundled pricing: Codex is included in ChatGPT Free, Go, Plus, Pro, Business, and Enterprise plans. Allowances vary by plan and model; the official pricing page publishes five-hour local-message limits and notes that additional weekly limits may apply. Users who exhaust an allowance can buy credits on eligible plans or run additional local tasks with an API key

at standard API rates. This mirrors Claude Code’s subscription bundling more closely than a pay-per-token CLI tool like Aider.

Configuration: Config lives at `~/.codex/config.toml`, covering model selection, sandboxing behaviour, and approval policy for shell commands.

Agentic capabilities: Codex CLI can read and edit files, run shell commands with configurable approval levels, and iterate autonomously across multi-step tasks, broadly comparable in scope to Claude Code and Gemini CLI.

Strengths: - Current GPT-5.6 Sol/Terra/Luna choices let you trade capability against how quickly you consume the plan allowance - Fast, low-overhead Rust binary - Genuine local-model mode via `--oss` (Ollama and LM Studio) – not just a cloud-only tool - Bundled into existing ChatGPT subscriptions, so many users already have access with no incremental cost - Very large and fast-growing community (high-five-figure/low-six-figure star range)

Weaknesses: - Best experience is tied to OpenAI’s own models and ChatGPT plans; using it as a purely model-agnostic tool is a secondary use case - Newer than Aider/OpenCode as a CLI-first product, despite OpenAI’s scale - Model availability and usage allowances change quickly, making static comparisons easy to age

3.8 Amp (Sourcegraph)

What it is: Amp is Sourcegraph’s hybrid CLI-plus-cloud coding agent. It pairs a local CLI with cloud-hosted “Orbs” for offloading longer-running or parallel work. Orbs are remote machines where agents can run without supervision; they pause after inactivity and are billed by the minute.

Author and licence: Sourcegraph.

Architecture: Amp combines a local CLI with cloud-side Orbs for longer-running or parallel work. Sourcegraph heritage informs the product, but this survey does not assume that every Sourcegraph code-intelligence feature is available inside Amp. The CLI handles local interactive work; Orbs handle cloud-side execution.

LLM provider support: Multi-model, with supported provider-key configuration and plugin/skill/hook extensibility. It is flexible, but “model-agnostic” overstates the default curated product surface.

Cost model: Amp has a free Hobby tier (pay-as-you-go orbs, no token fees or limits), a \$20/month Individual tier (45,000 orb minutes, unlimited repos), a Teams tier (pooled credits, no extra platform charge on top of members’ own plans), and a custom-priced Enterprise tier

(pooled credits only, plus SCIM/audit logs/IP allowlisting) (as of 2026-09-12). Individual and non-enterprise workspaces do not pay a model markup; Orb compute is metered separately by the minute with automatic pause for idle instances. Check the live pricing page before budgeting: Amp’s model and compute rates are usage-based and have changed structure before.

Strengths: - Sourcegraph-backed product lineage and cloud execution for longer tasks - Hybrid local/cloud model gives a path to offloading long-running or parallel tasks without leaving the CLI workflow - Supported multi-model routing with extensibility via plugins, skills, and hooks - No markup on individual pay-as-you-go pricing

Weaknesses: - Orb compute and agent inference are separate meters, so a subscription is not an all-in fixed-cost plan - Newer product identity means less accumulated community track record than Sourcegraph’s older Cody product - Cloud “Orbs” component means it is not a purely local/offline tool in the way Aider or Goose can be

3.9 Deprecated and Unverified Tools

3.10 Mentat

What it is: Mentat was a Python-based CLI coding tool and an early entrant in the space, taking a conversational approach to code editing with explicit context management.

Status: Fully archived and read-only since January 7, 2025 – this is a firm end-of-life, not a gradual slowdown. Note for anyone searching for current status: “Mentat” is now also used by an unrelated GitHub bot product from Abante AI, which can make search results confusing – the archived coding CLI and the Abante AI bot are different products that happen to share a name.

Note: For new users, Aider is a better choice in the same category – more active, more features, larger community.

Roo Code (fork of Cline) – also deprecated: Roo Code, a fork of Cline created in 2024 to move faster on multi-agent workflows (its “Boomerang Tasks” feature) and broader model support, was archived on May 15, 2026 and is now read-only. The project has publicly directed users to alternatives such as Cline and ZooCode. There is no live “current tool” entry for Roo Code in this document – by the time of this refresh it is itself historical.

3.11 Qwen Code and Kimi CLI

What they are: These are CLI coding agents from Chinese AI labs – Qwen Code from Alibaba’s Qwen team and Kimi CLI from Moonshot AI. Both are classified alongside Claude Code and Aider as CLI tools by Artificial Analysis’s taxonomy. Both are positioned as competitive alternatives particularly for users who want model diversity or prefer models strong at certain programming languages.

Provider flexibility: Qwen Code uses Qwen models (strong at code, particularly multilingual/non-English codebases; current coding-focused models include `qwen3-coder-next`, `qwen3-coder-plus`, and `qwen3-coder-480b`). Kimi CLI uses Kimi’s models with a very long context window.

Note: These tools were not covered by this round’s research refresh. Treat specific capability claims with appropriate scepticism until you test them directly.

4 IDE Extensions and Hybrid Tools

4.1 Cline (formerly Claude Dev)

What it is: Cline is an open-source autonomous coding agent that operates primarily as a VS Code extension. It is model-agnostic, OpenRouter-compatible, and one of the most capable agentic tools available for IDE-based workflows. It has since expanded well beyond VS Code and beyond the IDE entirely, moving it toward hybrid territory.

Author and licence: Open source at github.com/cline/cline. MIT licence.

Community size: Roughly low-60k GitHub stars (late-July 2026 snapshot). This makes it one of the most popular tools in the entire AI coding agent space, not just this subcategory.

Platform reach: Beyond VS Code, Cline runs in JetBrains IDEs and can connect to ACP-compatible editors such as Zed and Neovim through its CLI. It is now a real terminal agent as well as an IDE extension; the CLI requires Node.js 20+ and supports interactive, headless, and JSON-output workflows.

Architecture: Cline runs as a VS Code extension and has deep access to the VS Code API – language servers, the file system, the integrated terminal, and the browser (via Puppeteer integration). It operates in two modes:

- **Plan mode** – Cline analyses the task and produces a plan before executing
- **Act mode** – Cline executes changes directly

This Plan/Act split allows you to review the strategy before any files are modified.

LLM provider support: Cline is model-agnostic. It supports:

- Anthropic (Claude family)
- OpenAI (GPT-4o, o1)
- Google (Gemini)
- AWS Bedrock
- Vertex AI
- OpenRouter (verified – listed on OpenRouter’s works-with-openrouter page)
- Ollama for local models
- LM Studio
- Any OpenAI-compatible API endpoint

OpenRouter setup:

In VS Code: Open Cline extension settings

API Provider: OpenRouter

API Key: sk-or-...

Model: anthropic/claude-sonnet-5 (or any OpenRouter model)

Agentic capabilities: Cline can:

- Read and edit any file in the workspace
- Execute shell commands in the integrated terminal
- Create and delete files/directories
- Use the browser (view pages, click, type, screenshot)
- Run searches across the codebase
- Read error output and iterate automatically

Browser use is a standout capability – Cline can actually open a browser, navigate to a URL, interact with a web page, and read the results. This enables tasks like: “go to this API docs page and implement the integration.”

MCP support: Cline supports MCP servers. Configuration is via the Cline settings JSON. This means your existing MCP server investments (filesystem servers, database servers, custom tools) are portable to Cline.

Extensibility: Beyond MCP, current Cline has project rules and skills under `.cline/`, lifecycle hooks, plugins, and a system-prompt override. Older `.clinerules` guidance may still appear in documentation, but `.cline/rules/`, `.cline/skills/`, `.cline/hooks/`, `.cline/plugins/`, and `.cline/mcp.json` are the current CLI configuration surfaces.

CLI mode: The standalone CLI makes Cline usable outside any IDE. It supports plan mode, provider/model selection, MCP management, plugins, hooks, schedules, a background

hub, ACP integration, and structured JSON output. Windows support remains a separate question in the current installation documentation, so check the platform requirements before standardising on it.

Cost model: Open source. You can use Cline’s hosted provider, a direct provider/API key, or local Ollama/LM Studio; price and quota depend on the selected path. OpenRouter usage is billed by the underlying model.

Team note: Cline remains independent, open source, and actively developed as of end-July 2026.

Roo Code lineage: Roo Code forked from Cline in 2024 to push further on multi-agent workflows (Boomerang Tasks) and broader model support. It was archived on May 15, 2026 and is now read-only; most users evaluating that lineage now compare Cline and ZooCode.

Strengths: - Extremely capable agentic tool – one of the best - Model-agnostic with OpenRouter support verified - Browser use capability is notable among open-source tools - MCP support - Plan/Act mode for controlled execution - Very large and active community - Now reaches far beyond VS Code (JetBrains, ACP-compatible editors, and CLI)

Weaknesses: - The CLI and extension expose different workflows, and platform support is not uniform - Browser use capability means it can have unintended side effects if not supervised - Less scriptable than pure CLI tools

4.2 Continue.dev

What it is: Continue.dev is an open-source AI coding assistant for VS Code and JetBrains IDEs. It combines autocomplete, chat, and agent capabilities in a single extension, with deep configurability via a YAML config file. Since September 2025 it also ships a headless CLI, so it is no longer purely IDE-integrated.

Author and licence: Open source at github.com/continuedev/continue. Apache 2.0 licence.

Architecture: Continue.dev is structured around three core features:

1. **Autocomplete** – inline code completions as you type
2. **Chat** – conversational interface about code, with codebase context
3. **Agent** – agentic mode that edits files based on instructions

Configuration lives in `~/.continue/config.yaml` and can specify different models for different features (e.g., a fast local model for autocomplete, a more capable cloud model for agent tasks).

CLI mode (cn): Continue.dev shipped a headless `cn` CLI in September 2025, offering both a TUI interactive mode and a scriptable/background mode (`cn -p "prompt"` for one-shot invocation, plus `async` background jobs). This is a supplementary capability, not a pivot away from the IDE extension – Continue.dev is still primarily used as an IDE assistant, with the CLI extending it into scripts, CI, and terminal-first workflows.

LLM provider support: Continue.dev has one of the broadest provider lists:

- Anthropic, OpenAI, Google
- OpenRouter
- Ollama (confirmed at `localhost:11434` by default, remote connections via `apiBase`)
- LM Studio, `llama.cpp`
- Groq, Together AI, Replicate
- Self-hosted models via any OpenAI-compatible endpoint

Ollama setup (verified):

```
# ~/.continue/config.yaml
models:
  - title: "Local coding model"
    provider: ollama
    model: qwen2.5-coder:7b # verify the current Ollama model tag
    # For remote Ollama:
    # apiBase: http://192.168.1.100:11434
```

Agent mode: Continue.dev’s agent mode allows it to make multi-file edits based on instructions. Tool support depends on the underlying model – models must support function/tool calling. Note: some models that claim tool support may not work reliably in agent mode (this was partially corroborated in the research, though the specific claim was refuted as too broad).

Extensibility: Beyond MCP, Continue.dev supports a block system where community-built extensions (“blocks”) add new context providers, tools, and model configurations. This is effectively a plugin marketplace for the IDE assistant workflow.

Strengths: - Best-in-class for IDE users who want full local LLM support - Works in both VS Code and JetBrains - Highly configurable via `config.yaml` - Autocomplete + chat + agent in one package - OpenRouter and Ollama support confirmed - Headless `cn` CLI extends it into scripts, CI, and terminal workflows - Open source, Apache 2.0

Weaknesses: - Still primarily IDE-integrated; the CLI is supplementary, not a terminal-first redesign - Agent mode quality depends heavily on the model’s tool-calling capabilities - Less capable at long agentic chains than Cline or Claude Code - JetBrains support is less mature than VS Code

5 Dedicated AI IDEs

5.1 Cursor

What it is: Cursor is a VS Code fork with deep AI integration baked into every layer of the editor. It has become one of the most widely adopted commercial AI coding tools, with a large user base and reported ARR in the hundreds of millions.

Company: Anysphere, Inc. Private company, significant VC backing.

Pricing:

- **Hobby** – Free – limited completions and requests per month
- **Pro** – \$20/month – usage-based credit pool (not a fixed request count)
- **Pro+** – \$60/month – larger credit pool for heavier usage
- **Ultra** – \$200/month – highest credit pool, priority access to frontier models
- **Teams** – \$40/user/month standard or \$120/user/month Premium – team features, admin controls, and expanded usage/features
- **Enterprise** – custom

Billing moved from a fixed “500 fast requests/month” allowance to a usage-based credit pool in mid-2025 – treat any “N requests/month” framing of Cursor pricing as obsolete, not merely dated.

Architecture: Cursor extends VS Code with AI capabilities at multiple layers: inline completions, a chat sidebar, and Agent mode. It maintains a shadow workspace where it can test proposed changes before applying them. Current Cursor supports multiple parallel agents and cloud/background workflows; exact concurrency and branch limits are product- and plan-dependent, so avoid treating a fixed agent count as a durable limit.

LLM models available: Cursor Pro gives access to current-generation Claude, GPT, and Gemini models, alongside Cursor’s own in-house model, Composer. Cursor model/version details change frequently; check Cursor release notes for the exact current model lineup.

Terminology note: “Composer” and “Agent mode” are two different things, not interchangeable. Composer is the name of Cursor’s own in-house model – Anysphere trains and ships it as one of the model options in the picker. Agent mode is the autonomous multi-file execution feature: it accepts a task, proposes a plan, and executes changes across multiple files, and can run terminal commands in Yolo mode (automatic execution without prompting). Agent mode can be pointed at Composer or at any of the other bundled models.

Context mechanisms:

- `@codebase` – full codebase semantic search
- `@docs` – pull in documentation from URLs
- `@web` – web search integration
- `@file`, `@folder` – explicit file/folder references
- `@git` – reference git history and diffs

Rules: Current Cursor rules live primarily in `.cursor/rules/*.mdc` and global rules. `.cursorrules` is a legacy format retained for migration, not the preferred format for new projects.

Current Cursor pricing documentation also lists MCPs, skills, and hooks on its paid plans. Treat plan eligibility and implementation details as volatile, but do not describe Cursor as having only rules-file customisation.

Provider flexibility: Limited. While Cursor allows “BYO API key” (bring your own Anthropic/OpenAI key), it is not natively designed for OpenRouter or arbitrary providers, and it has no local LLM support. The product is optimised for the models it bundles.

Strengths: - Polished, deeply integrated AI coding experience - Agent mode is very capable, and now supports parallel multi-branch agents - Semantic codebase indexing enables large-codebase context - Large community, extensive documentation, many tutorials - Tab completion is best-in-class

Weaknesses: - IDE-bound – not a CLI tool - Limited model flexibility – still not OpenRouter compatible, no local model support - Proprietary – you are dependent on AnySphere remaining a going concern and maintaining pricing - Heavier than a terminal tool – requires a full VS Code instance - Usage-based credit pricing makes monthly cost less predictable than the old flat-request model

5.2 Windsurf (now Devin Desktop, Cognition AI)

What it is: Windsurf was an AI-first IDE built by Codeium, positioned as a direct competitor to Cursor. Its standout feature was the Cascade agent, which used “Flow” – a system that maintained contextual awareness across an entire coding session rather than treating each interaction as independent. The product has since been acquired and rebranded; treat “Windsurf” as the legacy name and “Devin Desktop” as the current one.

Ownership timeline: Cognition announced its acquisition of Windsurf on **July 14, 2025**; the transaction amount is not established by the current first-party sources used here. The product was subsequently presented as **Devin Desktop**. Treat Windsurf as the legacy name

and Devin Desktop as the current product name, but verify the current migration and feature terminology before relying on old Cascade documentation.

Company: Cognition AI.

Pricing: Devin Desktop has its own pricing ladder, separate from Cognition’s cloud Devin product (see the Devin entry under Cloud Platform Agents – do not conflate the two):

- **Free** – limited quota
- **Pro** – \$20/month
- **Max** – \$200/month
- **Teams** – \$80/month base + \$40/seat

Architecture: Devin Desktop remains a VS Code-derived desktop IDE. Its current product page describes agents running on the local machine and integrations such as extensions, plugins, and MCP; that does not establish local LLM inference.

LLM models: Devin Desktop uses Cognition-managed inference by default, with some degree of model selection in higher tiers.

Provider flexibility: Cognition-managed inference is the default; the current product page does not establish a general Ollama, LM Studio, or OpenRouter path. Treat local execution of an agent as distinct from running the model locally.

Strengths: - Local-machine agent execution and a polished IDE experience - Historically cheaper tiers than Cursor at the low end - Autocomplete remains strong (inherited from Codeium’s original product) - Clean, polished IDE experience

Weaknesses: - IDE-bound - Two ownership changes and a rebrand in under a year – evaluate stability before committing long-term - Less model flexibility than Cursor - Smaller community than Cursor; documentation is still catching up to the rebrand - Easy to confuse with Cognition’s separate cloud Devin product, which has different pricing and a different execution model

5.3 Zed AI

What it is: Zed is a high-performance code editor written in Rust, designed for speed and minimal latency. Its AI integration (Zed AI) brings Claude as the default model into this fast editor experience.

Architecture: Zed is not a VS Code fork – it is a ground-up implementation in Rust with native GPU acceleration. This makes it significantly faster than Electron-based editors (VS Code, Cursor, Devin Desktop). AI features are integrated via an assistant panel. Zed’s

Agent Client Protocol (ACP, launched January 2026) is a distinct interoperability feature none of the other dedicated AI IDEs offer: it lets Claude Code, Codex CLI, Gemini CLI, and OpenCode run directly inside Zed as pluggable agents, rather than requiring Zed’s own agent to do everything.

LLM models: Claude is the default. The supported provider list is now extensive: OpenAI, Gemini, Amazon Bedrock, DeepSeek, GitHub Copilot, LM Studio, Mistral, Ollama, OpenRouter, and Vercel.

Pricing: Zed the editor is free and open source. Zed AI is billed separately on token-based usage, restructured into three tiers:

- **Personal** – free permanently – 2,000 accepted edit predictions/month
- **Pro** – \$10/month – unlimited edit predictions, \$5 of tokens included, overage billed at API list price + 10%
- **Business** – \$30/seat/month

Strengths: - Extraordinarily fast editor – the best performance of any AI-integrated editor
- Open source editor core - Genuinely broad provider flexibility, including local (Ollama) and OpenRouter – no longer just “supports configuring alternative backends,” both are confirmed
Yes - ACP lets you bring your preferred CLI agent (Claude Code, Codex CLI, etc.) into the editor instead of relying solely on Zed’s own agent

Weaknesses: - Smaller plugin/extension ecosystem (incompatible with VS Code extensions)
- Native agentic mode is still less mature than Cursor’s or Devin Desktop’s - Windows is supported alongside macOS and Linux - Token-based overage billing can be less predictable than a flat monthly fee for heavy users

5.4 Google Antigravity

What it is: Google Antigravity is Google’s agent-first development platform. It includes Antigravity CLI, the Antigravity 2.0 desktop application, IDE integrations, an SDK, and managed agents in the Gemini API. It is the announced migration target for individual Gemini CLI users.

Company: Google (Cloud/Developer Products, the same organisation behind Gemini CLI – not Google DeepMind).

Pricing and access: Antigravity has a \$0 individual tier with basic weekly rate limits, Google AI Pro with higher limits and a flexible AI-credit pool, and Google AI Ultra with the highest quotas and access to third-party models. Google Cloud customers can use Anti-

gravity through the Gemini Enterprise Agent Platform with consumption-based API pricing. Antigravity’s plans page says there is no BYOK or custom endpoint for additional account quota, although the CLI can separately use a Gemini API key for direct Gemini-compatible inference.

LLM models: The current Antigravity product pages list Gemini models, Claude Sonnet and Opus 4.6, and `gpt-oss-120b` among the available models; exact availability depends on plan and surface. Managed Antigravity agents are powered by Gemini 3.7 Flash by default and can be configured to use supported Gemini 3.x models.

Why it matters here: Antigravity is now the relevant Google alternative for both terminal-first and desktop workflows. Antigravity CLI is a Go-based terminal tool with skills, hooks, subagents, and plugins; Antigravity 2.0 is a standalone desktop application for synchronous and asynchronous multi-agent work. The products are new and still evolving, so verify the current plan, model, and regional availability before standardising on them.

Strengths: - Direct migration path for users displaced by Gemini CLI’s consumer-account transition - Multi-model product surface, including Gemini, selected Claude models, and open-weight models - Under active development by Google

Weaknesses: - Newest entrant in this table – least battle-tested - Product and pricing details remain volatile across surfaces - No general BYOK/OpenRouter path for extending account quotas

5.5 AWS Kiro IDE and CLI

What it is: Kiro is AWS’s agentic development environment, available as a VS Code-based desktop IDE, terminal-native CLI, web surface, and mobile companion. These surfaces share one agent harness and the `.kiro/` project configuration. The IDE supports ordinary editor work plus structured Specs; the CLI adds headless and continuous-integration workflows.

Architecture and extensibility: Kiro combines project steering files, custom agents, skills, hooks, MCP servers, permissions, Powers, and checkpoints/rewind. It also discovers `AGENTS.md` files as steering context. Hooks can run shell commands or send prompts to the agent at events such as prompt submission, tool use, file changes, and agent lifecycle transitions. This makes Kiro one of the closest alternatives to Claude Code’s skills-plus-hooks model, although the file formats and event semantics differ.

Provider and model support: Kiro uses AWS-managed model routing rather than arbitrary provider endpoints or OpenRouter. Its current model selector includes AWS’s Auto route, Anthropic Claude models, OpenAI GPT-5.6 variants, and selected open-weight mod-

els. Free users get a limited monthly credit allocation; paid plans expose progressively larger credit pools and access to premium models.

Pricing (August 31, 2026): Kiro Free is \$0/month with 50 credits. Pro is \$20/month with 1,000 credits; Pro+ is \$40/month with 2,000; Pro Max is \$100/month with 5,000; and Power is \$200/month with 10,000. Paid users can buy add-on credits at \$0.04 each. Credits are fractional and model-dependent, so request counts are not a reliable substitute for the credit budget.

MCP: Kiro supports local stdio MCP servers and remote HTTP/SSE servers across its IDE, CLI, and web surfaces, with surface-specific differences. Treat remote-server transport and authentication as part of deployment testing rather than assuming every server works everywhere.

Strengths: - The strongest documented skills/hooks/MCP/steering story among the commercial IDE alternatives - Shared configuration across IDE, CLI, and web - Structured Specs for requirements, design, and task execution - CLI and ACP support, not just an editor extension - Clear credit-based pricing with a free tier

Weaknesses: - AWS-managed model routing; no OpenRouter or general BYO-provider path - Credit consumption varies by model and task, so heavy-use cost is not a flat subscription - Closed source and tied to AWS's product and model availability - Newer than the established IDEs; verify regional model access and feature availability

6 Cloud and Web Platform Agents

6.1 Grok Bot (xAI)

What it is: Grok Bot is xAI's commercial, hosted computer-use agent. xAI launched it in beta on **August 11, 2026** as a set of persistent AI teammates rather than as another chat session. A Bot has a name, a job, its own conversation and working context, and can turn a repeatable workflow into a skill or routine. Coding is one workload – xAI describes engineering Bots reproducing bugs and handing fixes to other Bots – but the product is aimed at work that crosses applications, websites, inboxes, documents, and business systems.

Architecture: The desktop and mobile applications are thin clients for chat, review, and approvals. The work runs on a persistent cloud computer with a browser, filesystem, and terminal. Bots can operate applications and websites directly, including services without a clean API or Model Context Protocol (MCP) integration. This makes Grok Bot materially different from an API-only agent: it can use the same graphical interfaces a human uses.

All Bots belonging to one user share that user’s cloud computer, including its files, browser sessions, and app logins; separate Bots are therefore not separate security boundaries.

Agentic workflow: You message a Bot like a colleague, give it a task and access to the required tools or files, and let it work asynchronously. Bots can remember stable preferences, run recurring routines, collaborate through handoffs, and return when they need approval. The product is closer to a persistent operations teammate than to a terminal REPL. It is not a drop-in replacement for Claude Code’s local repository workflow: its execution environment is hosted, and its strongest differentiator is cross-application computer use rather than terminal-native editing.

Access and cost (September 9, 2026 snapshot): Grok Bot launched for eligible SuperGrok and Cursor subscribers and later expanded to SuperGrok, paid Cursor, and Cursor Teams plans; enterprise rollout is handled through the Cursor account team. xAI’s August 26 announcement says Bot usage is separate from existing Grok and Cursor usage, while the current team documentation describes plan-specific allowances. Treat access, quotas, and billing as volatile and check the live plan matrix before budgeting. Grok Bot is a product-level service; access to Grok models through the xAI API is a separate integration path.

Security and privacy: Consequential actions can pause for approval, with Auto Review available to inspect computer and tool actions before execution. Passwords, passkeys, two-factor codes, CAPTCHAs, and payment confirmations are handed back to the user through computer takeover rather than entered into ordinary chat. Grok Bot requires cloud data storage and does not support Cursor’s Legacy Privacy Mode. Optional execution on the user’s local computer is separate from the shared cloud computer and asks for approval by default. Start with read-only tasks, least-privilege connectors, and explicit approval boundaries for sending, publishing, purchasing, deletion, permission changes, and production changes.

Strengths:

- Persistent cloud execution that continues while you are away
- Direct browser and desktop-style interaction across applications, including systems without APIs or MCP servers
- Durable Bots, routines, memory, and Bot-to-Bot handoffs for repeatable operational work
- Approval and credential handoff controls for consequential or sensitive actions

Weaknesses:

- Proprietary, hosted, and dependent on xAI/Cursor product access and policy

- Requires cloud storage; the shared computer means one Bot’s files and sessions can be available to the user’s other Bots
- Not a local-model, self-hosted, or OpenRouter-oriented workflow
- Beta product with rapidly changing plan eligibility, quotas, and administrative controls

6.2 OpenHands (formerly OpenDevin)

What it is: OpenHands is a highly capable open-source AI software engineer designed to solve complex software engineering tasks autonomously. It is the open-source project that most directly competes with commercial cloud agents like Devin.

Author and licence: MIT licence. github.com/OpenHands/OpenHands – the project migrated from the All-Hands-AI org to OpenHands; the new path is now the canonical one.

Community size: Roughly high-70k to low-80k GitHub stars and 7,000+ commits (late-July 2026 snapshot). One of the largest open-source AI coding projects by community size.

Architecture: OpenHands runs in a Docker container with a sandboxed execution environment. This is the key architectural difference from CLI tools – it doesn’t run directly on your machine; it runs in an isolated environment with full access to the shell, filesystem, and browser within that sandbox. This makes it safer for autonomous long-running tasks but adds Docker as a dependency. The project has recently restructured its core: the agent code now lives in separate repos (**software-agent-sdk**, **agent-canvas**), and the primary interface is the browser-based **Agent Canvas** rather than the older “Web UI” branding.

Interfaces:

- **Agent Canvas** – the primary interface, a browser-based UI for interacting with the agent
- **CLI** – a lightweight binary available at github.com/OpenHands/OpenHands-CLI
- **API** – programmable access for CI/CD integration

Installation:

Via Docker (recommended; the registry's default tag tracks the current runtime)
`docker pull docker.all-hands.dev/all-hands-ai/runtime`

Lightweight CLI binary (for terminal-first use)
`pip install openhands-cli`

LLM provider support: OpenHands supports multiple LLM providers via litellm, including Anthropic, OpenAI, Google, and others.

MCP support: OpenHands supports MCP server configuration across three transport types – SSE, Streamable HTTP, and stdio:

```
{
  "mcpServers": {
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"]
    },
    "notion": {
      "url": "https://mcp.notion.com/mcp"
    }
  }
}
```

For stdio servers, a proxy approach (SuperGateway/FastMCP) is recommended over direct stdio for reliability. Note: the claim that OpenHands “natively discovers MCP tools automatically” remains refuted – MCP is configured explicitly, not auto-discovered.

Strengths: - Very capable at complex, long-horizon tasks – closest open-source equivalent to Devin - Docker sandboxing means it can’t accidentally break your host system - Full MCP support across three transport types - Massive community and active development - Agent Canvas is more accessible than CLI tools

Weaknesses: - Requires Docker – heavier than CLI tools - Primarily a cloud/web platform – less suitable for quick interactive sessions - Slower to start up than CLI tools (Docker container spin-up) - Not a terminal-first tool despite having a CLI option

6.3 Devin (by Cognition AI)

What it is: Devin was the first high-profile “AI software engineer” capable of autonomously solving complex GitHub issues end-to-end. It is a commercial cloud product from Cognition AI.

Pricing: Devin now has Free (\$0), Pro (\$20/month), Max (\$200/month), Teams (\$80/month minimum plus \$40/month per full seat), and custom Enterprise plans. Paid self-serve plans include daily and/or weekly usage allowances; extra usage is purchased as on-demand credits at API pricing. ACU terminology is legacy for self-serve but remains relevant to Enterprise billing. This makes the headline price easier to understand, but heavy usage can still exceed the subscription price.

Devin now sits alongside a sibling product, **Devin Desktop** (formerly Windsurf, rebranded June 2026 after Cognition’s acquisition of Windsurf) – see the Windsurf/Devin Desktop entry for that product’s separate pricing ladder. The two are distinct: Devin is the cloud autonomous-agent product with plan quotas and on-demand credits; Devin Desktop is an IDE and agent command center.

Strengths: - Very capable for long-horizon autonomous tasks - Well-integrated with GitHub workflows - No longer prohibitively priced for individuals at light-to-moderate usage levels

Weaknesses: - Usage allowances and on-demand credits still make heavy-use costs less predictable than a simple flat subscription - Fully proprietary – no control over the stack - Not a CLI tool

7 Commercial AI Coding Assistants

7.1 GitHub Copilot with Agent Mode

What it is: GitHub Copilot is Microsoft/GitHub’s AI coding assistant. It now spans autocomplete, chat, an in-IDE “Agent Mode,” and a separate cloud-based “Copilot Coding Agent.” These are two distinct products, not one: **Agent Mode** (VS Code, synchronous, runs locally, GA since early 2026) executes edits directly in your editor session; the **Copilot Coding Agent** (cloud, asynchronous, GA September 2025) takes a GitHub issue assignment and opens a pull request without an open editor session. Both superseded the original “Copilot Workspace” preview, which was sunset May 30, 2025 – that name no longer refers to a current product.

Pricing (restructured with usage-based billing, June 1, 2026):

- **Free** – \$0/month; 2,000 completions/month
- **Pro** – \$10/month; unlimited completions plus included AI credits; additional consumption varies by model and workload
- **Pro+** – \$39/month; unlimited completions + \$70/month AI credits; premium model access including Claude Opus
- **Individual Max** – \$100/month; larger included AI-credit allowance; additional consumption varies by model and workload
- **Business** – \$19/user/month
- **Enterprise** – \$39/user/month

LLM provider: Copilot was never as OpenAI-locked as it appeared. The built-in model picker includes Claude, Gemini, Kimi, GPT, and other models, with availability varying by client and plan. GitHub now documents two BYOK paths: local BYOK in VS Code, JetBrains, Xcode, Copilot CLI, the Copilot app, and the SDK; and enterprise-managed custom models for Business/Enterprise. Local BYOK can work without a Copilot subscription and can target local or external providers, while enterprise BYOK is server-managed and remains in preview. BYOK and model availability vary by client and feature; code completions

remain on their separate billing path and are not automatically covered by every BYOK configuration.

OpenRouter support: Possible through a compatible local-BYOK/provider integration, but do not present it as one uniform Copilot feature. Client, plan, and organization policy determine whether a user can configure it; the enterprise custom-model path is still in public preview.

Strengths: - Deep GitHub integration – issues, PRs, code review - Real provider flexibility now, not just a wide built-in model picker - \$10/month remains the most affordable commercial completions tier - Enterprise features and security compliance

Weaknesses: - BYOK coverage depends on the client, feature, plan, and organization policy; local BYOK and enterprise-managed custom models are different paths - Agent Mode and Copilot Coding Agent are two separate mental models to learn, with different sync/async execution semantics - Less suited for non-GitHub workflows

7.2 Amazon Q Developer (formerly CodeWhisperer) – IDE transition to Kiro

What it is: Amazon Q Developer is AWS’s AI coding assistant, with deep integration into the AWS ecosystem (rebranded from CodeWhisperer in 2024). Its agentic mode autonomously implements features, refactors, and proposes multi-step changes in both IDE and CLI – this is no longer a completions-only tool, contrary to earlier characterisations.

CLI agentic mode: Supports Claude Sonnet 4 / 3.7 / 3.5 via `/model`, with file read/write, bash execution, and multi-step autonomy – a genuine general-purpose coding agent, not an AWS-console-only assistant.

Pricing:

- **Free** – 50 agentic requests/month
- **Pro** – \$19/user/month; 10,000 inference calls (~1,000 user requests)

Important – deprecation in progress: AWS will discontinue support for Amazon Q Developer IDE plugins and paid subscriptions on April 30, 2027; new Q Developer account and subscription signups closed May 15, 2026. Amazon Q Developer in the AWS console and other first-party AWS experiences is not covered by that IDE sunset. **AWS Kiro is the designated successor for IDE/CLI development** (see above) – do not recommend Q Developer as a new IDE investment without flagging this scope and date.

Strengths: - Best-in-class for AWS-heavy workflows - Security vulnerability scanning, IAM

policy generation - CLI agentic mode is now genuinely general-purpose, not AWS-console-limited

Weaknesses: - Being wound down – new signups already closed - AWS-focused – less useful for non-AWS codebases - Provider choice limited to Claude via Bedrock/CLI; no OpenRouter or broad model-agnosticism

7.3 JetBrains AI Assistant

What it is: JetBrains AI Assistant is integrated into IntelliJ IDEA, PyCharm, WebStorm, GoLand, and all other JetBrains IDEs. It provides chat, completion, and an autonomous multi-step agent called **Junie** – worth naming explicitly, since it is JetBrains’ direct answer to agentic coding assistants rather than a generic “agentic features” bundle.

Pricing:

- **AI Free** – \$0; 3 AI Credits per 30 days
- **AI Pro** – Individual \$10/month with 10 credits; Business \$20/month with 20 credits
- **AI Ultimate** – Individual \$30/month with 35 credits; Business \$60/month with 70 credits
- **AI Enterprise** – custom

All Products Pack, dotUltimate, and some IDE licences can include AI entitlements. AI Credits are the quota unit; top-up credits are available on eligible individual/business plans and expire after 12 months. External models and agents can also be used without a JetBrains AI subscription, but feature coverage differs.

LLM models and agents: JetBrains documents current Claude, Gemini, OpenAI, and other model choices, plus BYOK and local providers such as Ollama, LM Studio, and llama.cpp. Junie, Claude Agent, Codex, Copilot, and ACP-compatible agents can be used from JetBrains IDEs; exact availability depends on IDE version and account.

Strengths: - Best choice if you are already heavily invested in JetBrains IDEs - Deep IDE integration (refactoring, inspections, test generation) - Junie provides genuine autonomous multi-step agent capability, not just chat - Local model support now confirmed, closing a prior gap

Weaknesses: - Junie is less proven than Cursor’s agent mode or Cline on complex multi-file tasks - Pricing and entitlements vary between standalone AI plans, IDE bundles, and external-agent paths, adding cost-tracking overhead - JetBrains IDEs remain heavier than VS Code

7.4 Tabnine

What it is: Tabnine is an AI coding assistant with a long history (one of the first serious AI coding tools), historically focused on fast, high-quality inline completions. It has since pivoted toward agentic workflows.

Pricing (materially higher than before – the old ~\$12/month Pro tier was sunset in 2025):

- **Code Assistant** – \$39/user/month
- **Agentic Platform** – \$59/user/month; adds MCP integration, autonomous workflows, CLI access
- **Enterprise** – custom, with self-hosted model option

Ownership and LLM provider: Tabnine announced its acquisition by Tricentis on July 30, 2026. Tabnine uses its own models plus third-party models (Anthropic, etc.); provider-hosted inference can add variable token charges and a handling fee, while Enterprise still supports deployment options for data sovereignty.

Strengths: - Very fast completions (latency-optimised) - Enterprise self-hosted option – best for organisations with strict data policies - Agentic Platform tier now offers MCP integration and CLI access, closing the gap with agent-first tools

Weaknesses: - Pricing jumped sharply (\$12 → \$39/\$59/month) – no longer the budget completions option it once was - Agentic capability is newer and less proven than Cursor, Cline, or Claude Code - Being overtaken by Cursor/Windsurf on raw completion quality

7.5 Supermaven – discontinued, folded into Cursor

What it is: Supermaven was a VS Code-focused autocomplete tool with a very large context window for completions (300k tokens), notable for speed and accuracy.

Status: Supermaven is no longer an independent product. It was **acquired by Anysphere (Cursor’s parent company) in November 2024**, not by Cursor itself as a product – Anysphere absorbed the team and technology, and Cursor has integrated Supermaven’s completion engine into its tab-completion feature. If Supermaven was your tool of choice, Cursor is its direct successor; there is no standalone Supermaven product to evaluate separately.

8 Cross-Cutting Analysis

8.1 Provider Flexibility Analysis

A critical factor for Claude Code users concerned about API costs is whether an alternative tool is model-agnostic – allowing you to route requests to cheaper models, local models, or whichever provider has the best current pricing.

8.1.1 OpenRouter Compatibility

OpenRouter is a unified API gateway that provides access to hundreds of models from dozens of providers under a single API key, with pay-per-token pricing and no monthly subscription.

Tool	OpenRouter Support	Notes
Aider	Yes (verified)	Full setup at aider.chat/docs/llms/openrouter.html via litellm.
Cline	Yes (verified)	Listed on openrouter.ai/works-with-openrouter .
OpenCode	Yes (verified)	75+ providers incl. OpenRouter; do not use the prohibited Claude Pro/Max plugin route.
Continue.dev	Yes	Via OpenRouter provider in <code>config.yaml</code> .
Goose	Partial	Reachable via custom API endpoint config; not an explicitly named provider in Goose’s own docs.

Tool	OpenRouter Support	Notes
Amp (Sourcegraph)	Partial	Supported provider-key configuration; OpenRouter is not a named first-class integration in the current docs.
OpenAI Codex CLI	Partial	ChatGPT subscription is the native path; local via <code>--oss</code> ; OpenRouter reachable via API-compatible config, not a first-class integration.
Plandex	Partial	Via OpenAI-compatible endpoint.
SWE-agent	Partial	Typically direct Claude/OpenAI; OpenRouter-capable but not the primary pathway.
OpenHands	Partial	Via litellm provider abstraction; not a first-class named provider in docs surfaced this round.

Tool	OpenRouter Support	Notes
GitHub Copilot	Partial	Local BYOK exists in several clients; enterprise-managed custom models are a separate path. OpenRouter and feature coverage depend on client, plan, and policy.
GitHub Copilot CLI	Partial	Official BYOK covers only the <code>openai</code> , <code>azure</code> , and <code>anthropic</code> provider types (<code>openai</code> = any OpenAI Chat Completions-compatible endpoint); OpenRouter is not named as a supported provider, so it is reachable only unofficially via the generic <code>openai</code> -compatible path.
Grok Bot	No	Managed product; do not confuse xAI API access to Grok models with Grok Bot's product surface.

Tool	OpenRouter Support	Notes
Zed AI	Yes (was Partial)	Explicitly listed alongside 10+ other providers (Bedrock, DeepSeek, Copilot, LM Studio, Mistral, Ollama, Vercel).
Google Antigravity 2.0	No	Current plans do not offer BYOK/custom endpoints for extending account quotas; third-party model availability is product/plan-specific.
Cursor	No	BYO key for direct providers only.
Windsurf (→ Devin Desktop)	No	Cognition-managed inference; not OpenRouter-oriented.
Devin	No	Cognition-managed inference with plan quotas and on-demand credits.

Tool	OpenRouter Support	Notes
Gemini CLI	No	Locked to Gemini models; individual consumer access moved to Antigravity CLI.
Amazon Q Developer	No	Locked to Amazon/Anthropic via Bedrock.
AWS Kiro IDE	No	AWS-native model routing.
JetBrains AI Assistant	No	Tied to JetBrains' own model rotation, not user-configurable OpenRouter.
Tabnine	No	Own models plus a fixed provider list.

Conclusion: For OpenRouter flexibility, the open-source CLI tools (Aider, Cline, Open-Code, Continue.dev, and Goose) remain the clearest options. Amp supports provider keys but does not document OpenRouter as a named first-class integration. Zed AI also lists OpenRouter among its supported providers. OpenAI's Codex CLI supports local/OSS models via `--oss`, but OpenRouter is not its primary path. GitHub Copilot now has local BYOK in several clients and enterprise-managed custom models, but availability and feature coverage vary by client and plan. Cursor, Devin Desktop, Devin, and Grok Bot remain managed products rather than OpenRouter-style routing layers; Grok Bot's cross-application computer use should not be confused with provider flexibility.

8.1.2 Local LLM Support (Ollama / LM Studio)

Running models locally eliminates API costs entirely, but not hardware, electricity, or operational costs. Quality and hardware fit vary by model, quantization, context length, and runtime; validate the exact model/workload combination before standardising on it.

Tool	Ollama	LM Studio	Notes
Aider	Yes	Yes	Via litellm.
Cline	Yes	Yes	Native provider options.
OpenCode	Yes	Yes	Provider-agnostic architecture.
Continue.dev	Yes	Yes	localhost:11434 default.
Goose	Yes	Partial	On-device philosophy is core; MCP-configurable local providers.

Tool	Ollama	LM Studio	Notes
Amp (Sourcegraph)	No	No	Confirmed no local inference: Amp's Security Reference states inference always runs on Sourcegraph-hosted providers (Anthropic, OpenAI, Google, etc.); running the CLI locally is client/orchestration only. No official mention of Ollama, LM Studio, llama.cpp, or any local endpoint anywhere in Amp's docs (Cody, a separate Sourcegraph product, has experimental Ollama support – not applicable to Amp).
OpenAI Codex CLI	Yes	Yes	Via <code>--oss</code> flag; documented local providers are Ollama and LM Studio.

Tool	Ollama	LM Studio	Notes
Plandex	Partial	Partial	Via OpenAI-compatible API config.
OpenHands	Yes	Yes	Via litellm; Agent Canvas can be self-hosted.
GitHub Copilot	Yes	Yes	Local BYOK is documented for several clients, including Ollama and Microsoft Foundry Local integrations; coverage varies by client and feature.
GitHub Copilot CLI	Yes	No	Copilot CLI's own BYOK docs name Ollama explicitly as a local endpoint example under the <code>openai</code> provider type; LM Studio is not named there (it appears only in the separate Copilot app's BYOK docs) – reachable, if at all, only via the generic OpenAI-compatible endpoint path, unendorsed.

Tool	Ollama	LM Studio	Notes
JetBrains AI Assistant	Yes (was not listed)	Yes	Ollama, LM Studio, llama.cpp confirmed on IDEs v2025.1+.
Zed AI	Yes	Partial	Ollama explicitly supported.
Windsurf (→ Devin Desktop)	No	No	Current product documentation confirms local-machine agent execution, not local LLM inference.
Devin	No	No	Devin Cloud uses Cognition-managed inference; do not conflate local execution with local model support.
Grok Bot	No	No	Hosted computer-use product; xAI API model access is a separate path.
Cursor	No	No	Cloud models only.
Gemini CLI	No	No	Gemini-hosted models; individual consumer access moved to Antigravity CLI.
Amazon Q Developer	No	No	AWS/Bedrock-locked.

Tool	Ollama	LM Studio	Notes
AWS Kiro IDE	No	No	AWS-native.
Tabnine	No	No	Cloud/on-prem/air-gapped deployment options exist, but not consumer local-model tools like Ollama.
Google Antigravity 2.0	No	No	Google-managed models and quotas; the current plans page says no BYOK or custom endpoint for additional quota.

Best tools for local LLMs: Aider, Cline, OpenCode, Continue.dev, Goose, and JetBrains AI Assistant remain the clearest options for local model use. OpenAI’s Codex CLI supports local models through `--oss` with documented Ollama and LM Studio providers. Amp’s local CLI is not evidence of local inference; its current documentation describes hosted models and provider keys. Devin Desktop’s local-machine execution should not be treated as local model support without explicit provider documentation.

Recommended local models for coding (late-July 2026 snapshot):

- **Current small coding models** – choose from the model catalog supported by Ollama or LM Studio; exact model names, quantizations, and hardware requirements change quickly
- **Hardware** – treat 16GB VRAM as a workload- and quantization-dependent starting point, not a guarantee. Test the model at the context length and latency your workflow requires.

8.2 Feature Comparison Matrix

The matrix spans 17 tools, too many to render legibly as one table at this page width – split below into five groups.

Group 1: Claude Code, Aider, OpenCode, Cline

Feature	Claude Code	Aider	OpenCode	Cline
Interface	CLI	CLI	TUI	VS Code / CLI
Agentic file editing	Yes (full)	Yes (full)	Yes (full)	Yes (full)
Multi-file context	Yes	Yes	Yes	Yes
Shell execution	Yes	Yes	Yes	Yes
Web search	Yes	No	Partial	Yes
Browser use	Via MCP/WebFetch‡	No	No	Yes
MCP support	Yes (native)	No	Yes	Yes
Custom hooks	Yes (skills+hooks)	No	Plugins / skills / hooks	Skills / hooks / plugins
OpenRouter	No	Yes	Yes	Yes
Local LLMs	No	Yes	Yes	Yes
Open source	No	Yes (Apache-2.0)	Yes (MIT)	Yes (MIT)
Git integration	Yes	Yes (auto-commit)	Yes	Yes
Cost model	Max plan or API	API only	API or subscription	Hosted provider, API, or local
Self-hosted option	No	N/A	N/A	N/A

Group 2: Continue.dev, Goose, Gemini CLI, OpenHands

Feature	Continue.dev	Goose	Gemini CLI	OpenHands
Interface	VS Code / JetBrains	CLI + Desktop	CLI	Web / CLI

Feature	Continue.dev	Goose	Gemini CLI	OpenHands
Agentic file editing	Yes (partial)	Yes (full)	Yes (full)	Yes (full)
Multi-file context	Yes	Yes	Yes (1M tokens) [†]	Yes
Shell execution	Yes (agent mode)	Yes	Yes	Yes (sandboxed)
Web search	No	Yes	Yes	Yes
Browser use	No	No	No	Yes
MCP support	Partial	Yes (native) [§]	Yes	Yes (config)
Custom hooks	config.yaml	Extensions	Settings	Limited
OpenRouter	Yes	Partial	No	Partial
Local LLMs	Yes	Yes	No	Yes
Open source	Yes (Apache-2)	Yes (Apache-2)	Yes (Apache-2)	Yes (MIT)
Git integration	Yes	Yes	Yes	Yes
Cost model	API only	API/provider keys	Enterprise, API, or Vertex; consumer status disputed [¶]	API only
Self-hosted option	N/A	N/A	N/A	Yes

Group 3: Cursor, Windsurf, Codex CLI, Amp

Feature	Cursor	Windsurf	Codex CLI	Amp
Interface	IDE	IDE (now Devin Desktop)	CLI	CLI + Cloud
Agentic file editing	Yes (full)	Yes (full)	Yes (full)	Yes (full)
Multi-file context	Yes	Yes	Yes	Yes (agent context)
Shell execution	Yes (yolo mode)	Yes	Yes	Yes
Web search	Yes	Yes	Yes	Yes

Feature	Cursor	Windsurf	Codex CLI	Amp
Browser use	No	No	No	No
MCP support	Yes	Yes	Yes	Yes
Custom hooks	MCP / skills / hooks	Limited	Plugins / skills / hooks	Plugins / skills / hooks
OpenRouter	No	No	Partial	Partial
Local LLMs	No	No	Yes	No
Open source	No	No	Yes	No
Git integration	Yes	Yes	Yes	Yes
Cost model	\$20-200/mo + API	Free / \$20 / \$200 / Teams	Bundled ChatGPT	Free / \$20 Individual (orb minutes) / Teams+Enterprise pooled credits
Self-hosted option	No	No	No	No

Group 4: Devin, Grok Bot, Copilot

Feature	Devin	Grok Bot	Copilot
Interface	Cloud / Web UI	Cloud desktop/mobile app	IDE + Cloud
Agentic file editing	Yes (full, sandboxed)	Yes (computer-use)	Yes (full)
Multi-file context	Yes	Yes (persistent cloud computer)	Yes (Enterprise)
Shell execution	Yes (sandboxed)	Yes (cloud computer)	Yes
Web search	Yes (agentic browser tool)¶	Yes (browser/search)	Yes
Browser use	Yes	Yes	No
MCP support	Yes	Yes (connectors/custom MCP)	Yes

Feature	Devin	Grok Bot	Copilot
Custom hooks	Limited	Routines / skills / Auto Review	Limited
OpenRouter	No	No	Partial
Local LLMs	No	No	Yes
Open source	No	No	No
Git integration	Yes	Yes (via cloud computer)	Yes
Cost model	Free / \$20 / \$200 + usage	Eligible plan + separate Bot usage	\$10-100/mo indiv.
Self-hosted option	No	No	No

Group 5: Kiro, Antigravity

Feature	Kiro	Antigravity
Interface	IDE + CLI + Web	Desktop + CLI + IDE integrations
Agentic file editing	Yes (full)	Yes (full)
Multi-file context	Yes	Yes
Shell execution	Yes	Yes
Web search	Yes	Yes
Browser use	No	Yes
MCP support	Yes (full)	Yes
Custom hooks	Hooks + steering + skills	Hooks + skills + plugins
OpenRouter	No	No
Local LLMs	No	No
Open source	No	No
Git integration	Yes	Yes
Cost model	\$20-200/mo (credits)	Free (individuals)
Self-hosted option	No	Partial (SDK/enterprise paths)

Product details vary by plan or surface. † Gemini CLI’s 1M-token context window is a model-level feature; product quotas still apply. ‡ Claude Code’s browser access is via MCP servers (e.g. Playwright) or WebFetch, not a built-in browser. § Goose’s MCP support is a

confirmed core architectural pillar with documented extensions and broad MCP ecosystem compatibility. ¶ Google’s transition announcement and current Gemini CLI authentication pages conflict on consumer access; verify the sign-in path before standardising on it. Grok Bot’s browser, MCP, and computer-use capabilities are product-level features; do not infer equivalent capabilities for the xAI API from this row. ¶ Devin’s web search is not a separately branded feature – it is the agentic cloud browser tool proactively looking up documentation and solutions during a task. This is distinct from “Devin Search,” a separately documented codebase-search feature; do not conflate the two.

8.3 Extensibility Deep Dive

One of Claude Code’s most powerful features is its extensibility system: Skills (reusable, discoverable workflow packages that can be invoked directly or selected by the agent), hooks (commands that fire on events like tool calls or session start), and MCP servers (external tools exposed via a standard protocol). This combination allows you to build a highly personalised, automated coding workflow. How do the alternatives compare?

8.3.1 Claude Code’s Extension Architecture

For context, Claude Code’s three-layer system works as follows:

1. **Skills** – Markdown files stored in `~/.claude/skills/` that define reusable workflows. When invoked as `/skill-name`, their content is loaded and executed. Skills can include checklists, decision trees, and instructions for specific types of tasks (debugging, code review, PR creation).
2. **Hooks** – Shell commands configured in `settings.json` that fire on specific Claude Code events. For example: run linting before a commit, post a notification when a session ends, validate that tests pass after edits. Hooks are executed by the Claude Code harness, not by the AI – this makes them reliable and deterministic.
3. **MCP Servers** – External processes exposing tools via the Model Context Protocol. Claude Code can call these tools as part of its reasoning (read from a database, query an external API, access browser dev tools, etc.). Any MCP server in the ecosystem works.

8.3.2 How Alternatives Compare

Aider: No equivalent to skills or hooks. No MCP support. Configuration via `.aider.conf.yml` covers model settings and preferences but not custom workflows. *Least extensible of the serious alternatives.*

OpenCode: MCP support confirmed. Its current plugin system can add custom tools, hooks, and integrations, and its configuration supports agent skills. The model and file formats differ from Claude Code, so portability is not automatic, but OpenCode now belongs in the serious extensibility comparison. *Mid-range extensibility, with a stronger automation story than the previous edition credited it with.*

Cline: MCP support confirmed. Current Cline adds project rules and skills, lifecycle hooks, plugins, schedules, and ACP support around the Plan/Act workflow. Its configuration is not compatible with Claude Code’s skills or hook event schema, so migration still requires adaptation. *One of the closest alternatives for programmable local workflows.*

Continue.dev: The “blocks” system provides a plugin-like architecture for adding new context providers and tools. Config-driven customisation is deep. No hook system. MCP support via tool configuration. *Good for IDE users, weaker hook/automation story.*

Goose: MCP is a core architectural pillar, not a bolt-on – Goose’s extension system is built directly on MCP, with documented extension support and broad MCP ecosystem compatibility. It is a strong conceptual match for Claude Code’s MCP layer, though its format differs and its hook/skill semantics are not drop-in compatible. *Strong extensibility story.*

Gemini CLI: MCP, extensions, and hooks are documented. Individual consumer access is disputed across Google’s current pages and the transition announcement; Antigravity CLI is the announced migration target and has skills, hooks, subagents, and plugins. Enterprise/API users should verify which features are present in their Gemini CLI release.

OpenHands: MCP support confirmed, with three transport types (SSE, Streamable HTTP, stdio) – a proxy approach (SuperGateway/FastMCP) is recommended over direct stdio for reliability. Configuration is required; MCP tools are not auto-discovered. Limited hook/automation equivalent otherwise. *Weakest extensibility story among the actively-developed alternatives.*

Grok Bot: Supports built-in connectors and custom MCP connectors, alongside skills, routines, Bot handoffs, and approval rules. MCP is optional rather than foundational: the Bot can use a browser and hosted computer directly when a service has no MCP server. Its routines and Auto Review controls are product-specific and are not compatible with Claude

Code hooks or other agents’ skill formats.

8.3.3 The AGENTS.md Open Standard

Alongside MCP, a second cross-cutting convention has emerged that matters for anyone comparing extensibility stories: **AGENTS.md**, an open standard for project-level agent instructions. It was released in August 2025 and donated to the Linux Foundation’s Agentic AI Foundation in December 2025. OpenAI’s current announcement says it has been adopted by more than 60,000 open-source projects and agent frameworks, including Amp, Codex, Cursor, Devin, Factory, Gemini CLI, GitHub Copilot, Jules, and VS Code. This is an adoption claim from OpenAI, not an independent census. Claude Code itself can consume it through `CLAUDE.md` imports.

The distinction from MCP matters: MCP standardises *tool/capability* portability (how an agent reaches external systems); AGENTS.md standardises *project-instruction* portability (how an agent learns a repo’s conventions, build steps, and constraints). They’re complementary, not competing – a tool can support one, both, or neither. For anyone currently maintaining a pile of per-tool rule files (`.clinerules`, `.cursorrules`, `CLAUDE.md`), AGENTS.md is worth tracking as a potential single source of truth that many tools can read natively, reducing the duplication of instructions across tools.

8.3.4 What This Means for Migration

If you have heavily invested in Claude Code’s extensibility system (skills, hooks, MCPs), a complete migration requires rebuilding your workflow on whatever alternative you choose. The good news: MCP server investments are portable to any MCP-compatible tool (Cline, OpenCode, Gemini CLI, Goose, OpenHands, Amp, Antigravity, and Kiro). OpenCode, Cursor, Cline, Antigravity, and Kiro now have their own skills/hooks/plugin surfaces, but formats and event semantics differ. The hooks system is still the hardest part to reproduce faithfully – alternatives require their own plugin model, shell wrappers, or git hooks. If your per-tool instruction files (rather than MCP servers) are the bigger migration cost, look at consolidating onto AGENTS.md – it is now adopted across tens of thousands of projects, and Claude Code can import it into `CLAUDE.md`.

8.4 Cost Analysis

If you are a heavy Claude Code user currently on a Max plan, what would the same usage cost on each alternative? Note first that “unlimited usage” is not an accurate description of the Max plan: Max 5x (\$100/month) and Max 20x (\$200/month) both carry a five-hour

session limit and a separate weekly limit, with further caps and model-specific effects possible. Anthropic’s live help page is the authority for the current limits; do not convert them into a fixed message count because message capacity depends on model, context, and workload.

8.4.1 Defining “Heavy Usage”

For this analysis, heavy usage means: - ~4-8 hours of active coding per day - Typical session: long context (50k-200k tokens input), many file edits, running tests - Estimated: ~100-200 million input tokens per month, 10-20 million output tokens per month

These are rough estimates – actual token consumption varies enormously by workflow and model. The worked examples below use the midpoints (~150M input / ~15M output tokens per month).

8.4.2 API Cost Estimates (Direct Provider, as of August 31, 2026)

Pricing changes frequently; treat these as ballpark figures using current published rates, not guarantees.

Anthropic Claude Sonnet 5:

- Input: \$2/million tokens (current standard price)
- Output: \$10/million tokens (current standard price)
- Heavy usage estimate: $(150\text{M} \times \$2 + 15\text{M} \times \$10) / 1\text{M}$ approximately **\$300 + \$150 = ~\$450/month**
- *This is why a Max plan (5-hour and weekly caps notwithstanding) remains good value for genuinely heavy users, even though it isn’t the “unlimited” plan the old framing implied.*

Pricing note: Anthropic’s current pricing page says the \$2/\$10 Sonnet 5 launch price is now standard; the previously announced September 1 increase will not occur.

Anthropic Claude Haiku 4.5:

- Input: \$1/million tokens
- Output: \$5/million tokens
- Heavy usage estimate: approximately **\$225/month**
- *Cheaper and faster than Sonnet 5, though the fastest/cheapest current Claude tier is no longer as dramatically cheap as the old Haiku 3 figures (\$0.25/\$1.25) this document previously cited – Anthropic’s tiering has shifted upward across the board.*

OpenAI GPT-5.6 Terra (current balanced tier):

- Input: \$2/million tokens
- Output: \$12/million tokens
- Heavy usage estimate: $(150\text{M} \times \$2 + 15\text{M} \times \$12) / 1\text{M}$ approximately **\$300 + \$180 = ~\$480/month**
- *GPT-5.6 Sol is the flagship at \$4/\$20 per million; GPT-5.6 Luna is the cost-sensitive tier at \$0.20/\$1.20. The model and price selected materially change the result.*

At the same 150M input / 15M output workload, the corresponding estimates are approximately **\$900/month for Sol** (\$600 + \$300), **\$480/month for Terra** (\$300 + \$180), and **\$48/month for Luna** (\$30 + \$18), before caching or any subscription allowance.

Google Gemini 3.1 Pro Preview (paid API tier):

- $\leq 200\text{k}$ context: \$2.00 input / \$12.00 output per million tokens
- $> 200\text{k}$ context: \$4.00 input / \$18.00 output per million tokens
- Heavy usage estimate: approximately **\$480/month** at $\leq 200\text{k}$ context, approximately **\$870/month** above it
- *Gemini 3.5 Flash (\$1.50/\$9.00) has a separate API free tier with reduced quotas, but heavy usage as defined here will exceed it. Gemini CLI's individual Google-account path moved to Antigravity CLI on June 18, 2026.*

DeepSeek V4 Flash (illustrative direct-API rates; gateway rates differ):

- Input: \$0.14/million tokens
- Output: \$0.28/million tokens
- Heavy usage estimate: **~\$25/month** at the stated rates and workload
- *Still dramatically cheaper than frontier models. Rates vary by provider and may include cache-hit, peak/off-peak, or gateway-specific pricing; this is an illustrative direct-API calculation, not a universal rate.*

Local model via Ollama or LM Studio:

- Cost: \$0 in API charges, plus hardware and electricity
- Hardware requirement: depends on model, quantization, context length, and runtime; validate the actual workload rather than relying on a generic VRAM threshold

8.4.3 Per-Tool Cost Summary

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Aider + Claude Sonnet 5	\$0 tool + API	~\$450/mo (\$2/\$10 standard pricing)	N/A	Illustrative calculation for the stated 150M input / 15M output workload.
Aider + Claude Haiku 4.5	\$0 tool + API	~\$225/mo	N/A	Cheaper same-vendor fallback; the “reduce cost, stay on Claude” scenario.
Aider + DeepSeek v4-flash (OpenRouter)	\$0 tool + API	Provider/model- dependent	N/A	Current gateway pricing can differ from direct DeepSeek pricing and may include cache or peak/off-peak rates.
Aider + a low-cost OpenRouter model	\$0 tool + API	Provider/model- dependent	N/A	Use the provider’s live rate card; do not treat a single gateway quote as universal.
Aider + Ollama (local)	\$0	\$0	GPU hardware cost only	Choose a currently supported local coding model and validate its quan- tization/context fit.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
OpenCode + DeepSeek v4-flash	\$0 tool + API	~\$25/mo	N/A	OpenCode's current provider path supports DeepSeek; this estimate uses the current V4-Flash direct rates before any gateway markup.
OpenCode (ChatGPT- native)	\$0 tool + ChatGPT subscription	Bundled – see Codex CLI row (same OpenAI subscription tiers apply since the Jan 2026 OpenAI partnership)	N/A	New row.
Cline + OpenRouter (model varies)	\$0 tool + API	Varies by provider/model	N/A	Route cheaply via OpenRouter; use current provider pricing.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Amp (Sourcegraph)	Free (Hobby, pay-as-you-go) or \$20/mo (Individual)	Individual \$20/mo includes 45,000 minutes of orb time; Teams and Enterprise are pooled-credit, no fixed per-seat platform charge (as of 2026-09-12)	No	Self-hosted is confirmed unavailable at any tier, including Enterprise (which adds SCIM/audit logs/IP allowlisting but stays SaaS). No markup over provider API prices for individual/non- enterprise plans; orb compute is metered separately by the minute with auto-pause.
Gemini CLI (individual)	Not applicable	Not applicable after the June 18, 2026 consumer transition	N/A	Use Antigravity CLI for individual Google-account access. Gemini CLI remains available through enter- prise/API/Vertex paths.
Gemini CLI (teams/API)	Varies	Google Developer Program, AI Studio, and Vertex AI paths differ	N/A	Choose based on identity, privacy, quota, and billing requirements.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Google Antigravity 2.0	\$0 individual tier	\$0 individual tier with weekly limits; Google AI Pro/Ultra raise limits; enterprise is consumption-priced	No	Direct migration target for individual Gemini CLI users; exact limits and model availability vary.
Cursor Pro	\$20/mo	\$20/mo (usage-based credit pool, not a hard cap; heavy users may need Pro+/Ultra)	N/A	Old “500 fast requests/month” framing is gone; credit-pool billing since mid-2025.
Cursor Pro+	\$60/mo	\$60/mo (larger credit pool)	N/A	New tier.
Cursor Ultra	\$200/mo	\$200/mo	N/A	New tier.
Devin Desktop (formerly Windsurf) – Pro	\$20/mo	\$20/mo	No documented local inference	Current product name and pricing; local-machine execution is not local model inference.
Devin Desktop – Max	\$200/mo	\$200/mo	No documented local inference	Current tier.
Devin Desktop – Teams	\$80/mo base + \$40/seat	\$80 + \$40/seat	No documented local inference	Current tier.
Devin (cloud agent, Pro)	\$20/mo	\$20/mo plus on-demand credits at API pricing	N/A	Current plan; daily and weekly usage allowances apply.
Devin (cloud agent, Max)	\$200/mo	\$200/mo plus on-demand credits at API pricing	N/A	Current power-user plan with a larger weekly allowance and no daily cap.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Devin (cloud agent, Teams)	\$80/mo minimum + \$40/full seat	Shared on-demand credits; full seats include their own allowance	N/A	Replaces the legacy ACU/Core and \$500 team-plan descriptions.
Grok Bot	Eligible Super-Grok/Cursor/Teams plan	Plan-dependent plus separate Bot usage allowance	N/A	Beta product; access, quotas, and billing vary by plan and change quickly.
GitHub Copilot – Individual Free	\$0/mo	\$0 (2,000 completions/mo cap)	N/A	New tier detail.
GitHub Copilot – Individual Pro	\$10/mo	\$10 + variable consumption after included AI credits	N/A	AI-credit consumption varies by model and workload; code completions have a separate allowance.
GitHub Copilot – Individual Pro+	\$39/mo	\$39 + overage (\$70 credits included)	N/A	New tier, includes premium models (Claude Opus access).
GitHub Copilot – Individual Max	\$100/mo	\$100 + overage (\$200 credits included)	N/A	New tier.
GitHub Copilot – Business	\$19/user/mo	\$19/user/mo + policy-controlled model catalog	N/A	Unchanged from old doc.
GitHub Copilot – Enterprise	\$39/user/mo	\$39/user/mo	N/A	Unchanged from old doc.
GitHub Copilot CLI	Bundled – inherits Copilot plan tiers above	Same	N/A	No standalone pricing found; open question whether it requires a specific plan tier.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Amazon Q Developer – Free	\$0/mo	\$0 (50 agentic requests/mo cap)	N/A	New row.
Amazon Q Developer – Pro	\$19/user/mo	\$19/user/mo (10,000 inference calls, ~1,000 user requests)	N/A	Also flag the sunset (new signups closed May 15 2026, support ends April 30 2027, Kiro is successor).
AWS Kiro – Free	\$0/mo	\$0 (50 credits)	N/A	New row.
AWS Kiro – Pro	\$20/mo	\$20 (1,000 credits) + \$0.04/credit overage	N/A	New row.
AWS Kiro – Pro+	\$40/mo	\$40 (2,000 credits) + overage	N/A	New row.
AWS Kiro – Pro Max	\$100/mo	\$100 (5,000 credits) + overage	N/A	New row.
AWS Kiro – Power	\$200/mo	\$200 (10,000 credits) + overage	N/A	New row.
JetBrains AI – Free	\$0/mo	\$0 (3 credits/30 days)	N/A	Some IDE bundles include AI entitlements; external models/agents are also available.
JetBrains AI – Pro	\$10/user/mo individual; \$20 business	Included 10/20 credits per 30 days; eligible top-ups	N/A	AI Credits are the quota unit; bundle and organization rules vary.
JetBrains AI – Ultimate	\$30/user/mo individual; \$60 business	Included 35/70 credits per 30 days; eligible top-ups	N/A	AI Credits are the quota unit; bundle and organization rules vary.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Tabnine – Code Assistant	\$39/user/mo (annual)	\$39/user/mo plus provider-dependent inference/handling charges	N/A	Replaces old doc’s “~\$12/month Pro” – that tier was sunset in 2025.
Tabnine – Agentic Platform	\$59/user/mo (annual)	\$59/user/mo plus provider-dependent inference/handling charges	N/A	Adds MCP integration, autonomous workflows, and CLI access.
Zed AI – Personal	\$0/mo	\$0 (2,000 edit predictions/mo)	N/A	Replaces old doc’s vague “free tier and credits system.”
Zed AI – Pro	\$10/mo	\$10 + token overage at API list price +10% (\$5 tokens included)	N/A	New precise figure.
Zed AI – Business	\$30/user/mo	\$30/user/mo	N/A	New tier.
OpenAI Codex CLI	Bundled in ChatGPT plan	Free (\$0), Go (\$8/mo), Plus (\$20/mo), Pro (\$100/mo) – plan allowance and purchasable credits vary by tier; local <code>--oss</code> mode is \$0	\$0 via <code>--oss</code> (Ollama/LM Studio)	Usage limits and included credits change by plan.
OpenHands	\$0 tool + API	Varies by chosen provider/model	N/A	Docker overhead is a hardware/time cost, not a monthly fee.

8.4.4 Cost Migration Strategy

Scenario 1: Keep Claude-quality results, accept higher costs Use Aider or Cline with direct Anthropic API. Cost: roughly \$450/month for the heavy-usage workload defined here at Sonnet 5’s current standard rate. This is still the worst-case scenario financially – you lose the Max plan subsidy, though the gap is narrower than it was under 2025 Sonnet pricing.

Scenario 2: Reduce cost with model diversity Use Aider or OpenCode with OpenRouter, routing to a current low-cost model for routine tasks and Claude/GPT-5.6 for complex ones. Cost depends on the selected provider, model, and routing strategy; expect a quality trade-off on simpler models.

Scenario 3: Go local for routine work, cloud for hard tasks Use Aider or Cline with Ollama or LM Studio and a current local coding model, then escalate to a cloud API for complex multi-file refactors. Cost is near zero for routine work, plus occasional API costs for hard tasks.

Scenario 4: Fixed-cost IDE subscription Move to Cursor (Pro \$20/month, Pro+ \$60/month, or Ultra \$200/month) or Devin Desktop/Windsurf (Pro \$20/month, Max \$200/month). You lose terminal-native workflow; usage pools and on-demand usage mean the effective monthly cost is not necessarily fixed.

Scenario 5: Free-for-individuals alternative Antigravity CLI is the announced first-party migration path for individual Google-account users and has a \$0 tier with weekly quotas. Google’s current transition announcement conflicts with some Gemini CLI authentication pages, so test the live sign-in path before making it a dependency. API-key and Vertex use have their own quota, billing, and data-use terms.

9 Recommendations

9.1 If you want the closest CLI experience to Claude Code

Primary: Aider – mature, reliable, git-native, OpenRouter-supported, local LLM support. Start here. The absence of MCP and hooks is a real limitation if you’ve invested heavily in those, but the core editing workflow is the most Claude Code-like of any open-source tool.

Secondary: OpenCode – MCP support and broad provider flexibility (75+ connected providers) in a terminal tool with a very large user base. One caveat that changes its positioning: older bundled Anthropic plugin pathways are no longer bundled in current releases, while ChatGPT subscriptions work natively. If part of OpenCode’s appeal to you was Claude-adjacency through older plugin flows, verify your current setup before committing.

Worth evaluating if you want mainstream backing or hybrid local/cloud: OpenAI’s Codex CLI is a major CLI-native competitor, bundled into ChatGPT plans from Free through Enterprise, with local-model support via `--oss` for Ollama and LM Studio. Amp offers a hybrid local-CLI-plus-cloud-Orbs model with \$20/\$200 subscriptions or pay-as-you-go billing; model inference and Orb compute are separate meters.

9.2 If you want the best IDE-integrated alternative

Primary: Cline – model-agnostic, OpenRouter-supported, browser use, MCP-compatible, Plan/Act mode, and now available beyond VS Code through JetBrains and ACP-compatible editors, plus a full CLI. The most capable open-source agentic IDE/CLI tool available. Note: Roo Code, a 2024 fork of Cline that grew its own multi-agent following, was archived in May 2026; most of its userbase returned to Cline, reinforcing Cline’s position here.

Secondary: Continue.dev – best-in-class local LLM support and deep IDE integration across VS Code and JetBrains simultaneously, and as of September 2025 it also ships a headless `cn` CLI for scriptable, non-interactive use – useful if you want IDE-first but occasionally need a scriptable escape hatch.

Commercial option: Cursor (Pro \$20/month, Pro+ \$60/month, Ultra \$200/month) – if you want the best polished commercial experience and are comfortable with a VS Code-derived workflow. Note the terminology shift: “Composer” is Cursor’s own in-house model, not the agent mode – autonomous execution is called “Agent mode.” Current Cursor supports multiple parallel agents, but exact concurrency and branch limits are plan-dependent; it remains OpenRouter- and local-model-incompatible.

9.3 If cost is the primary constraint

Free option: Antigravity CLI has an official \$0 individual tier authenticated with a Google Account. It is the simplest zero-subscription starting point in this survey, though weekly quotas and Google’s model ecosystem still matter.

Ultra-cheap option: Aider or OpenCode with a current low-cost model via OpenRouter. DeepSeek V4 Flash is one candidate, but quote its current provider/model rate card at the time of purchase; the older `deepseek-chat/deepseek-reasoner` API names are sunset.

Zero API cost: Any Ollama- or LM Studio-compatible tool plus a local coding model – hardware and latency depend on the model, quantization, context, and workload.

A caution on Devin: Devin’s \$20/month Pro plan includes usage allowances, but extra work is purchased as on-demand credits at API pricing. That is clearer than the legacy ACU

model, but it still means heavy users can spend beyond the subscription. Devin competes on autonomy and capability, not guaranteed low cost.

9.4 If you need persistent cross-application or asynchronous work

Primary: Grok Bot – if the task crosses websites, inboxes, documents, and business applications and can continue while you are away. Its persistent cloud computer and computer-use interface can reach systems that lack clean APIs or MCP servers. It is a poor fit when local repository control, local models, self-hosting, or strict no-cloud handling is the main requirement.

Coding-focused alternatives: Devin remains the better comparison for GitHub-centred autonomous software engineering, while OpenHands is the better comparison when you want an open-source, self-hostable cloud-agent stack. Claude Code remains the better fit for interactive terminal-native work.

9.5 If MCP server investments are critical

Tools with confirmed MCP support: Claude Code (native), OpenCode, Cline, Gemini CLI, Antigravity, Goose (MCP is a core architectural pillar with documented extensions and broad ecosystem compatibility), OpenHands (config, multiple transports), Amp, Kiro, and Grok Bot (connectors and custom MCP, alongside direct computer use). Of these, OpenCode, Cline, and Kiro have the strongest current combination of agentic execution and extensibility; Goose remains worth a closer look when on-device execution and MCP portability matter. Grok Bot is the better fit when MCP is unavailable but browser or desktop computer use can bridge the gap.

9.6 If you need extensibility similar to Claude Code’s skills+hooks

The honest answer: no alternative is drop-in compatible with Claude Code’s skills+hooks system. The closest approximations: - **Skills equivalents:** Cline skills, OpenCode skills, Kiro skills, AGENTS.md-style instruction files, and custom system prompts - **Hooks equivalents:** Cline hooks, OpenCode hooks, Kiro hooks, Git hooks, shell wrappers, and CI/CD tooling – event names and permissions differ - **MCP equivalents:** Cline, OpenCode, Gemini CLI, Goose, OpenHands, and Amp all support MCP – your server investments are portable - **Instruction-file equivalents:** if per-tool rule files (rather than MCP servers) are your bigger migration cost, AGENTS.md is now adopted across tens of thousands of projects and is a better consolidation target than any single vendor’s format

9.7 Migration strategy

Rather than a hard switch, a practical migration path:

1. **Set up Aider** as a Claude Code complement today. Get comfortable with it. It is free to try with Haiku (cheap) or Ollama (free).
2. **Test OpenCode** – its MCP support and provider flexibility make it a strong candidate for a full Claude Code replacement. Treat the old Claude Pro/Max plugin route as unsupported; use an Anthropic API key or another documented provider path.
3. **Test your free-tier fallback** – Antigravity CLI is free for individuals within product quotas. Run your own representative workload before relying on that allowance for daily work.
4. **Invest in OpenRouter** – get an API key. With OpenRouter, you’re never locked to a single model again. As model prices drop (historically, they do), your costs drop automatically.
5. **Protect your MCP investments** – build MCP servers in preference to tool-specific plugins wherever possible. MCP compatibility is growing across the ecosystem. **Do the same for AGENTS.md** if you maintain per-tool instruction/rule files (`.clinerules`, `.cursorrules`, `CLAUDE.md`) – it is now adopted across tens of thousands of projects and is a lower-maintenance way to keep project instructions portable than maintaining one file per vendor.

10 References

First-party sources checked on **August 31, 2026** for the main survey snapshot. Grok Bot sources were checked on **September 9, 2026**:

- [Anthropic Max plan and usage limits](#)
- [Claude Code with Pro or Max](#)
- [Anthropic API pricing](#)
- [OpenAI Codex pricing](#)
- [OpenAI API models and pricing](#)
- [Codex repository and current CLI distribution](#)
- [Google’s Gemini CLI to Antigravity CLI transition announcement](#)
- [Gemini CLI authentication](#)
- [Gemini API pricing](#)
- [Antigravity plans and pricing](#)
- [Antigravity overview](#)

- [Antigravity CLI installation](#)
- [Antigravity MCP documentation](#)
- [Amp pricing](#)
- [Amp documentation](#)
- [Amp skills and plugins and plugin API](#)
- [Cline CLI reference](#)
- [Cline installation and supported surfaces](#)
- [Cline OpenRouter provider](#)
- [OpenCode providers and developer providers](#)
- [OpenCode repository](#)
- [Goose repository](#)
- [OpenHands MCP guide and MCP settings](#)
- [Cursor pricing and Cursor rules](#)
- [Devin Desktop](#)
- [Cognition's Windsurf acquisition announcement](#)
- [Devin pricing and self-serve billing](#)
- [Zed pricing and providers and Windows availability](#)
- [Kiro pricing and Kiro documentation](#)
- [Kiro steering, hooks, and MCP](#)
- [JetBrains AI plans and usage, supported models, and agents](#)
- [Tabnine pricing and Tabnine acquisition announcement archive](#)
- [GitHub Copilot BYOK and models/pricing](#)
- [Amazon Q IDE end-of-support scope](#)
- [DeepSeek API pricing and V4 release note](#)
- [Introducing Grok Bot](#)
- [Grok Bot overview](#)
- [Grok Bot approvals, security, and privacy](#)
- [Grok Bot for teams and enterprises](#)
- [Grok Bot access expansion](#)
- [Aider repository and Aider edit formats](#)
- [OpenRouter works-with-openrouter](#)
- [Continue Ollama guide](#)
- [AGENTS.md and OpenAI's Agentic AI Foundation announcement](#)

Secondary sources used only for background:

- [Artificial Analysis coding-agent taxonomy](#)
- [Awesome CLI Coding Agents](#)

- MorphLLM’s Claude Code alternatives comparison